基于可行集鲁棒定位与覆盖–路径联合优化的无线电干扰源快速定位与清除

摘　要

无线电干扰源的快速定位与清除是保障电磁频谱安全的重要手段，机器狗依托测向与清除动作在未知环境中自主完成这一任务。本文针对半径 1800 m 圆形区域内全向与定向混合干扰源的定位与清除问题，以集合成员定位为统一几何基础，建立了基于楔形交可行集的圆覆盖判别模型、基于最坏直径的鲁棒选点模型与覆盖–路径联合优化的在线调度模型，实现了全部干扰源的快速定位与清除。

针对问题一，首先将每次示向度观测转化为以测点为顶点、张角 2° 的楔形，经半平面交得到**定位可行集**；在此基础上用 **Thales 判据**检验指定直径对生成的圆能否覆盖该区域。检验环节做 **Monte Carlo 实验**（$N = 20000$）：1.4% 的样本至少存在一对不能覆盖区域的直径端点，并给出 $\alpha = 90^\circ$ 交会角下的精确临界构造。结论：直径圆**不一定覆盖**定位区域；若要求任一直径圆均覆盖须检验全部直径对，若只问是否存在一个可覆盖的直径圆则检验是否至少一对通过。

针对问题二，以第一次观测后的可行集 F 为对象，将第二测点对 F 内任意可能源都保证接收这一要求刻画为全部顶点 1000 m 接收盘的交集；进而以第二次观测后剩余可行域的**最坏直径** Φ 为鲁棒选点目标，该目标直接度量任意两个可能真值之间的最大辨识误差；并建立**两层网格粗筛、候选加密精排**的求解流程。算例求得最优第二测点 $\boldsymbol{S_2^* = (-727.8,\ -2.9)}$，加密复核下最坏直径 **111.7 m**，边界与内部联合采样显示该值稳定；结论限于离散样本意义。

针对问题三，将全向源清除建模为**感知覆盖与巡检路径联合设计**，形成三层递进的在线策略：覆盖层采用中心加 1000 m **正八边形九点布局**，解析证明边界最坏点距最近测点约 **956 m < 1000 m**，保证发现；定位层以半平面交与最小覆盖圆（MEC）收敛，并以 MEC 圆心补测（配对消融快 **10.6%**）；清除层用**最近邻加 2-opt** 路线，配合全局交错补测、带距离闸的收敛即清与跳测判定。离线 500 局回归**全部清除**、平均 **359.4 s/源**；正式测试 3 局 **40/40 全清**、平均 **367.1 s/源**。

针对问题四，对含定向源的混合场景建立**四层保障结构**：①发现层采用 **26 点环形骨架**，以可用测点方位**最大空隙判据**刻画发现条件；经**三条独立数值通道与四层 Lipschitz 闭合**，证明连续域无盲区，保守最小角裕度下界约 **3.1°**；②清除闸 MEC 半径 **≤ 19.75 m** 出手必中；③极端兜底层为 **17.2 m 三角细网格**，由夹逼不等式 $|s - G| \le 19.93 < 20$ m 保证候选网格中必存在可接收且可清除的测点，在线程序在时间允许时执行；④效率调度层。在线程序正式测试 3 局 **37/37 全清**、平均 **550.2 s/源**；布局选型经 400 局配对对照，**26 点环形**较 36 点方格快 **18.3%**。

真机实验合计 **33 局 440/440 源全清、零拒绝动作**，故障注入回归 100 项全部通过。

关键词： 集合成员定位；最小覆盖圆；Thales 判据；鲁棒优化；覆盖路径规划；在线调度

一、问题重述

1.1 问题背景

某区域无线电通信受到不明发射源干扰。干扰源分为全向与定向两类：全向源的可接收区域是以源为圆心、半径为接收半径的圆盘；定向源的可接收区域是以源为边界点、沿发射方向的半圆盘。接收半径存在不确定性，仅知其介于 1000 m 与 1500 m 之间。测向设备给出的示向度读数带有同一地点固定、全局有界 $\pm 1^\circ$ 的误差。干扰源分布在以参考原点为圆心、半径 1800 m 的圆形区域内，最多 20 个，占用信道号互不相同；清除半径为 20 m，机器狗移动速度 5 m/s，每次测量耗时 6 s。参赛程序通过模拟器接口依次提交测量、清除与退出动作，以清除全部干扰源所用的模拟器虚拟时间最短为目标。

1.2 问题提出

1. **问题一（定位区域的直径与圆覆盖性）**：两次示向度观测将源位置约束为一个平面区域。求该区域的直径，并回答以直径 r 为直径的圆能否覆盖此定位区域；若不一定，给出判别覆盖与否的方法。
2. **问题二（第二检测点的选择）**：第一次测量后，选择第二个检测点，使得在保证对可行集内任意可能源均能接收到信号的前提下，第二次观测后剩余定位区域尽可能小。建立选点模型，并对给定算例给出具体选点与相应区域大小。
3. **问题三（全向源自动定位与清除）**：设计可执行程序策略，在信道 1–20 上自动完成全部全向干扰源的发现、定位与清除，目标为总虚拟时间最短。
4. **问题四（混合源自动定位与清除）**：在问题三基础上加入定向源。定向源只有在测点位于其发射半平面内且距离不超过接收半径时才可能被发现，要求程序策略仍能完成全部源的发现、定位与清除。

二、问题分析

2.1 问题一分析

问题一的核心是刻画两次示向度观测下的定位区域，并判定以直径为直径的圆能否覆盖该区域。两站各带 $\pm 1^\circ$ 有界误差的示向度读数不再对应两条射线，而是两个楔形区域，源位置的严格可行集是全部楔形与目标圆域的交；凸多边形的直径必在顶点对上取得，而某一直径对生成的直径圆能否覆盖全区域，可由 Thales 定理给出逐顶点的代数判据。主要难点有二：一是口径的澄清，即是否引入源必在场地内这一先验会改变可行集本身，全文必须统一；二是数值实现中目标圆域只能以内接多边形逼近，临界实例的结论必须经得起该近似的误差传播检验。

2.2 问题二分析

在问题一楔形交可行集模型的基础上，问题二的任务是选择第二检测点，使定位尽可能收敛。该测点面临两层不确定性的叠加：源在可行集 F 内的位置未知，且第二次读数仍带 $\pm 1^\circ$ 误差。因此选点准则应是鲁棒的，即在最坏情形下（对 F 内任意源位置、任意容许误差）使第二次观测后的剩余区域直径最小。该目标在 S_2 上无解析表达式，需要在保证接收交集约束下做数值搜索；搜索的难点在于内层最坏值本身是对 G 与 e 的最大化，只能离散采样估计，故采样口径与收敛性复核必须明确交代。

2.3 问题三分析

在问题一、二定位模型的基础上，问题三的核心是全向源的自动搜索、定位与清除，其本质是有限检测点集的感知覆盖与巡检路径的联合设计。覆盖层回答的问题是测点放在哪里才能保证发现每一个源：全向源可接收区域为圆盘，故只需测点集合对目标圆域形成半径不超过接收半径下限 1000 m 的覆盖。路径层回答的问题是按什么顺序访问这些测点与补测点才能使总时间最短：由演练数据反推的虚拟时间构成显示移动占绝对主导（约 71%），因此调度的关键在于压缩巡视路程与补测移动量。此外，信道号互不相同与 `no_signal` 的三义性（空信道、超半径或覆盖外）意味着空信道只能靠逐点测量排除，这部分测量量不可压缩；定位与清除环节则需要明确的收敛闸门与失败上报语义。

2.4 问题四分析

在问题三搜索–定位–清除调度框架的基础上，问题四的核心是混合源场景下的自动定位与清除。定向源的加入改变了发现问题的性质：可检出条件从距离判据变为距离与位于发射半平面内的联合判据，测点集合必须对任意发射朝向都保证检出。这要求对每个可能源位置，其 1000 m 内可用测点的方位分布不存在半空缺，即方位空隙判据。发现之后的定位、清除与调度可沿用问题三框架，但需要补齐两个新环节：一是稀疏布局下补测管线能否吸收每源示向度减少的代价（布局选型问题），二是极端病态构型下补测用尽仍未收敛频道的兜底清除机制。

三、模型的假设

1. 测向误差为同一地点固定、全局有界的 $\pm 1^\circ$ 系统性偏差，不是随机噪声；据此采用集合成员估计而非统计滤波。
2. 接收半径仅知介于 1000 m 与 1500 m 之间；凡涉及保证的结论一律按最坏情形 1000 m 建立，1500 m 仅用于跳测判定等不安全即保守的方向。
3. 干扰源位置分布于目标圆域内，且占用信道号互不相同；同一信道至多一个源。
4. 机器狗移动速度 5 m/s、每次测量 6 s、清除耗时约 6 s，均由官方模拟器演练日志实测反推核实。

5. 清除只取决于机器狗与源的距离（$\le 20\,\mathrm{m}$），与源的类型和朝向无关。

6. 忽略机器狗转弯、加减速等未在协议中出现的运动学细节；测点坐标允许为目标圆域外的场外点（依据：附件 1 明确机器狗位置不构成约束）。

其中假设 1 与 2 直接决定全文的建模口径：误差有界使可行集为确定性集合，接收半径下限使保证发现类命题可以严格陈述。若实际误差分布偏离有界假设，可行集会相应扩大，发现保证的结论形式不变而数值裕度需要重估。

四、符号说明

表 1: 主要符号

符号	含义
$S_k,\ \theta_k$	第 k 个测点位置与其示向度读数
$\mathbf{u}(\theta)$	方位角 θ 的单位向量 $(\cos\theta, \sin\theta)$（东为 0°，逆时针为正）
W	全部示向度楔形之交（纯观测可行集）
$K,\ F$	引入场地先验后的可行集（$K = W \cap D_{1800}$；问题二中记 F）
$d,\ V_i$	凸多边形直径与顶点
CR	保证接收交集 $\bigcap_{V\in\mathrm{vert}(F)} \mathit{disk}(V, 1000)$
$K_2(G, e)$	源位于 G、第二次误差为 e 时第二次观测后的可行域
$\Phi(S_2)$	第二测点的鲁棒目标：K_2 最坏直径
r_{MEC}	可行集最小覆盖圆（MEC）半径
$G,\ \varphi$	干扰源位置与定向源发射朝向
$\Theta(G)$	G 的 $1000\,\mathrm{m}$ 内可用测点方位集合
ρ	三角格覆盖半径（$=\sqrt{3}/3$ 倍格间距）
C	信道集合 $\{1, \ldots, 20\}$；c 为单个信道

五、模型的建立与求解

5.1 问题一：定位区域的直径与圆覆盖判据

5.1.1 可行集的构造

第 k 站位于 S_k、读数 θ_k。由于误差有界 $\pm 1^\circ$，真实源 G 必须位于以 $\mathbf{u}(\theta_k \pm 1^\circ)$ 两条射线为边界的楔形内，用二维叉积写成两个线性不等式：

$$\mathrm{cross}\big(\mathbf{u}(\theta_k - 1^\circ),\, P - S_k\big) \ \ge\ 0 \ \wedge\ \mathrm{cross}\big(\mathbf{u}(\theta_k + 1^\circ),\, P - S_k\big) \ \le\ 0. \tag{1}$$

全部楔形的交记为 W。是否进一步取 $K = W \cap D_{1800}$（D_{1800} 为目标圆域）是口径问题：K 引入了源必在场地内这一新先验，是不同的可行集。当两站视线近平行时 W 无界，此时

纯观测口径只能报告直径不存在，转入 K 属于更换口径而非同一对象的另一种算法。本文问题一的主口径为 $K = W \cap D_{1800}$，数值上以 720 边形内接逼近 D_{1800}（弦高 0.0171 m）。

5.1.2 直径与覆盖的严格判据

可行集为凸多边形，其直径 $d = \max_{i,j}\|V_i - V_j\|$ 必在顶点对上取得。以直径对 (A, B) 为直径的圆覆盖全区域，当且仅当每个非端点顶点 V 都落在以 AB 为直径的圆内或圆上；由 **Thales 定理**，这等价于

$$\mathrm{slack}(A, B) \;=\; -\max_{V \neq A,B} (V - A)\cdot(V - B) \;\geq\; 0. \tag{2}$$

判据必须对每一个取得直径的顶点对逐一检验：可行集的直径对可以不止一对，此时每一对的直径圆都必须通过检验，任何一对失败即存在未被覆盖的可行点；直径最大本身不能推出覆盖。数值实现中点积量纲为 m^2，容差取相对量 `rel_tol`$\cdot d^2$。

量词口径说明：题目所问的以直径 r 为直径的圆能否覆盖此定位区域存在两种量词读法，分别对应两个不同的判据：(i) 若要求任一直径对生成的圆均覆盖可行域，则充要条件是全部直径对通过式 (2)（任一对失败即不覆盖）；(ii) 若问是否存在某个直径对生成的圆覆盖可行域，则充要条件是至少一对通过式 (2)。本文的 Monte Carlo 实验按逐对检验统计：1.4% 的样本存在未通过检验的直径对，这已足以否证直径圆必然覆盖定位区域，故直径圆**不一定**覆盖定位区域这一结论在两种量词读法下均成立。

5.1.3 式 (2) 充要性的证明

证明基于如下恒等式：设 $C = (A + B)/2$，则对任意点 V，

$$\|V - C\|^2 - \frac{\|A - B\|^2}{4} \;=\; (V - A)\cdot(V - B). \tag{3}$$

事实上，$(V-A)\cdot(V-B) = \|V\|^2 - V\cdot(A+B) + A\cdot B$，而 $\|V-C\|^2 = \|V\|^2 - V\cdot(A+B) + \|C\|^2$，两式相减得 $\|C\|^2 - A\cdot B = \frac{1}{4}\|A+B\|^2 - A\cdot B = \frac{1}{4}\|A-B\|^2$，即式 (3)。

于是 V 落在以 AB 为直径的圆内（含圆上） $\iff$ $\|V - C\| \leq \|A - B\|/2$ $\iff$ $(V - A)\cdot(V - B) \leq 0$。充分性：可行集是紧凸集，其直径必在某顶点对 (A, B) 上取得；若其余全部顶点满足式 (2)，则全部顶点落在该圆内，圆是凸集，顶点的凸包（即整个可行集）亦落在圆内，覆盖成立。必要性：可行集任一点都是顶点的凸组合，若存在顶点 V 违反式 (2)，该顶点本身是合法源位置且落在圆外，覆盖失败。故式 (2) 对单个直径对是充要的；对全部直径对逐一检验同样不可省略：直径对的全体是有限集，覆盖要求对每一对都成立，任一对失败即存在未被该直径圆覆盖的可行点。式 (3) 同时解释了余量的几何意义：slack 即半径平方与顶点到中点距离平方之差 $\|A - B\|^2/4 - \|V - C\|^2$（量纲 m^2），故数值容差取 `rel_tol`$\cdot d^2$ 与之同量纲匹配。

5.1.4 数值实验与结论

Monte Carlo 实验（$N = 20000$；交会角 $\alpha \in [20°, 160°]$；站距 $\in [200, 1500]$ m；误差 $e \sim U(-1°, 1°)$）结果：19715 例全覆盖，285 例（1.4%）存在不可覆盖的直径圆，0 例

真源落在可行集外（实现校验）。失败实例全部为 4 顶点，其交会角 25% ∼ 75% 分位为 $[90.3°, 91.8°]$，接近正交的交会角使四顶点近似内接于圆，这是失败样本均落在近矩形构型这一经验描述的来源，但它与式 (2) 不是逻辑等价，不能写成充要条件。图 1 给出全部 2 万样本的 Thales 余量分布：余量为负（不可覆盖）的样本集中出现在近正交构型的零线附近，而大交会角与小交会角区域的裕量都显著为正。

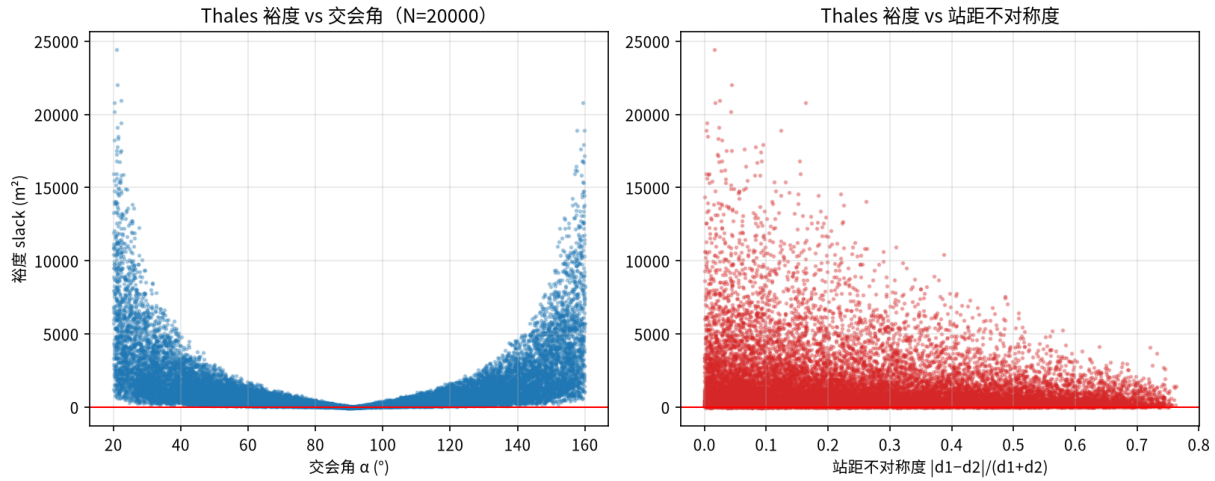


图 1: Thales 余量随交会角（左）与站距不对称度（右）的分布（$N = 20000$）：不可覆盖样本集中在近正交构型。

为说明临界性，构造精确鞍点：$\alpha = 90°$、$d_1 = d_2 = 1000\,\mathrm{m}$、源位于中垂线上，此时可行集四顶点恰好内接于其直径圆，Thales 余量 slack $= 0$；在反对称误差的一个小邻域内，一侧方向仍覆盖、相反方向余量翻负（表 2）。这说明覆盖性在临界构型两侧改变，必须逐实例检验，任何关于一般情形都覆盖的断言都不成立。

表 2: 临界鞍点构型（$\alpha = 90°$，$d_1 = d_2 = 1000\,\mathrm{m}$）对反对称误差 $\pm\varepsilon$ 的响应

ε (°)	$e_1 = +\varepsilon,\ e_2 = -\varepsilon$	slack (m^2)	$e_1 = -\varepsilon,\ e_2 = +\varepsilon$	slack (m^2)
0.0	覆盖	−0.000	覆盖	−0.000
0.05	覆盖	2.131	不覆盖	−41.767
0.1	覆盖	4.269	不覆盖	−39.499
0.2	覆盖	8.568	不覆盖	−34.986
0.3	覆盖	12.899	不覆盖	−30.506
0.5	覆盖	21.654	不覆盖	−21.641
1.0	覆盖	44.125	覆盖	0.000

三个算例的结果（详见附录支撑材料 `q1_results.md` 与图 2）：算例 1（常规交会）$d = 70.3\,\mathrm{m}$，覆盖；算例 2（三站）$d = 62.2\,\mathrm{m}$，覆盖；算例 3a（近平行视线）口径 ① 下 W 无界，口径 ② 下 $d = 1700\,\mathrm{m}$、两个直径对均不满足式 (2)，不可覆盖；算例 3b（1° 交

会角病态薄片）$d = 3001.7\,\mathrm{m}$。

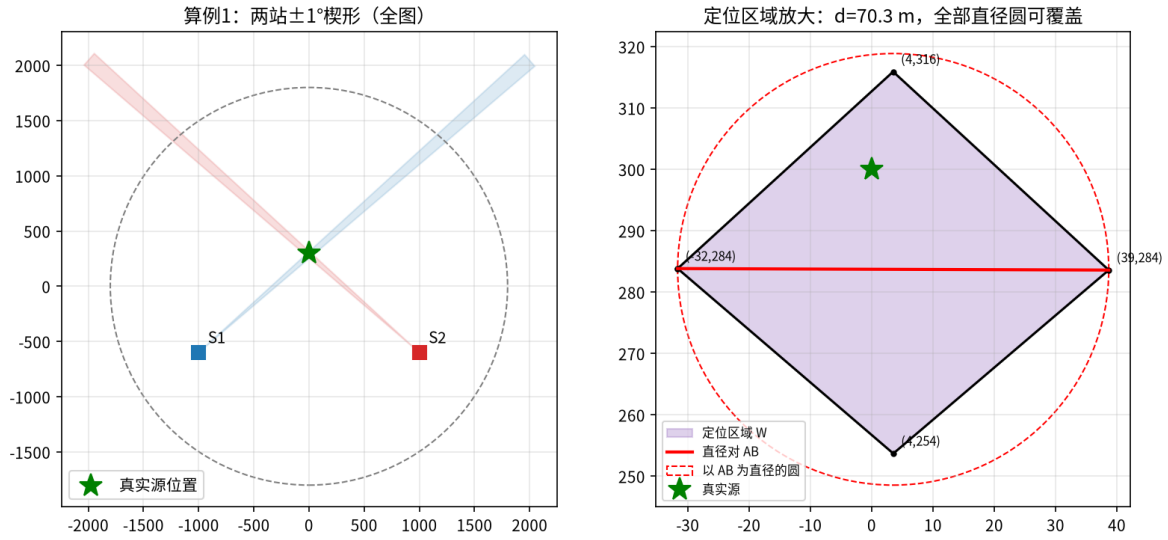


图 2: 问题一算例 1：楔形交可行集与直径圆（覆盖成立）。

问题一结论：以定位区域直径为直径的圆**不一定**覆盖该区域。判别方法为：枚举全部直径对，逐一按式 (2) 检验 Thales 余量；全部通过则覆盖，否则存在该直径圆遗漏的顶点，不覆盖。该结论的适用范围是凸多边形可行集（两站情形即楔形交），抽样统计仅用于说明失败构型的分布特征。

5.2 问题二：第二检测点的鲁棒选择

5.2.1 保证接收交集

第一次观测后，可行集为

$$F \;=\; W(S_1,\theta_1)\;\cap\; \mathit{disk}(S_1,1500)\;\cap\; D_{1800}, \tag{4}$$

其中 $\mathit{disk}(S_1,1500)$ 由接收半径上界给出。第二测点 S_2 要对 F 内任意可能源保证接收（距离不超过 1000 m 下限），等价于

$$S_2 \in CR \;=\; \bigcap_{V\in \mathrm{vert}(F)} \mathit{disk}(V,1000). \tag{5}$$

该等价性严格成立：对 F 内任意点 $G=\sum\lambda_i V_i$（凸组合）与任意 $S_2\in CR$，由三角不等式 $\|G-S_2\|\le\sum\lambda_i\|V_i-S_2\|\le 1000$。反之若 $S_2\notin CR$，则存在顶点 V 距 S_2 超过 1000 m，该顶点是合法源位置却不可保证接收。数值上各圆盘以 360 边形近似，得到的是保守内近似。

5.2.2 鲁棒目标与求解算法

第二测点的质量以第二次观测后的剩余区域度量。对源位置 $G\in F$ 与第二次误差 $e\in[-1°,1°]$，

$$K_2(G,e) \;=\; F\;\cap\; W\big(S_2,\;\mathrm{br}(S_2\to G)+e\big)\;\cap\; \mathit{disk}(S_2,1500), \tag{6}$$

其中 $\mathrm{br}(S_2 \to G)$ 为 S_2 指向 G 的方位角，$\mathit{disk}(S_2, 1500)$ 是第二测点自身可接收性的约束（实现中以 720 边形内接逼近，径向误差约 0.014 m）。鲁棒目标为

$$\Phi(S_2) \;=\; \max_{G \in F,\; e \in [-1^\circ, 1^\circ]} \operatorname{diam} K_2(G, e). \tag{7}$$

以直径而非 MEC 半径作为目标的原因：直径直接度量剩余可行域内任意两个可能真值之间的最大辨识误差，是定位不确定性的最坏尺度；MEC 半径只刻画以圆近似可行域时的清除可行性，作为清除判据在问题三、四中使用，二者分工不同。Φ 关于 G 的凸性未证明，内层最大值只能离散采样：边界取顶点加每边 8 个内点、误差步长 0.05°。外层采用轻量**两层网格搜索**：粗网格（方位 3°× 半径 40 档）筛选，前 25 名候选局部细化（$0.5^\circ \times 5\,\mathrm{m}$），前 5 名再以全量加密样本（边界加密一档，另在 F 内部按面积加权均匀采样 3 万点）逐一复核排序。因此报告值的意义是加密离散样本上的最坏值的最小者，不构成连续域的严格最优证明。计算量方面，粗网格 $120 \times 40 = 4800$ 个候选中仅 25 个进入细化、5 个进入加密复核，内层采样支配耗时；粗筛只承担排序功能，最坏值一律以加密复核为准，避免了粗网格离散误差进入最终报告。

5.2.3 算例结果

算例设定 $S_1 = (-800, -1000)$，$\theta_1 = 53.5^\circ$，真源 $(0, 100)$。结果：

- 最优候选 $S_2^* = (-727.8, -2.9)$（方位 133.5°，$\rho^* = 543.3\,\mathrm{m}$），加密复核最坏直径 $\Phi_{\mathrm{dense}} = \mathbf{111.7\,m}$，为一次测量区域尺度（1500 m）的 **7.4%**；
- 收敛性复核：边界样本加密一档后差 0.00 m；考虑 Φ 凸性未知，另在 F 内部按面积加权均匀采样 3 万点（误差步长 0.05°），内部最坏值 111.7 m 未超过边界值，结论在边界与内部联合采样下稳定；
- S_2^* 与第一示向度的垂直方向（144°/324°）偏差 10.0°：与适度距离上垂直交会的定性认识一致，但受 CR 截断与网格离散影响，不构成垂直最优的证明；
- 对照：S_2 位于第一示向度线上时交会几何退化，修正口径下定位区域被 F 截断为细条，$\Phi = 750.1$ m（有限但定位效果很差）；无保证接收约束的 $\rho = 1500$ m 垂直点 $\Phi \approx 54$ m，仅作对照，说明保证接收约束付出了代价。

表 3: 加密复核下前 5 个候选第二测点的鲁棒目标值（最优解附近目标面平坦）

S_2 (m)	方位 (°)	ρ (m)	Φ_{dense} (m)
(−727.8, −2.9)	133.5	543.3	**111.717**
(−734.7, −2.6)	134.0	548.3	111.721
(134.8, −646.0)	333.0	548.3	111.721
(−721.0, −3.3)	133.0	538.3	111.733
(−741.7, −2.4)	134.5	553.3	111.745

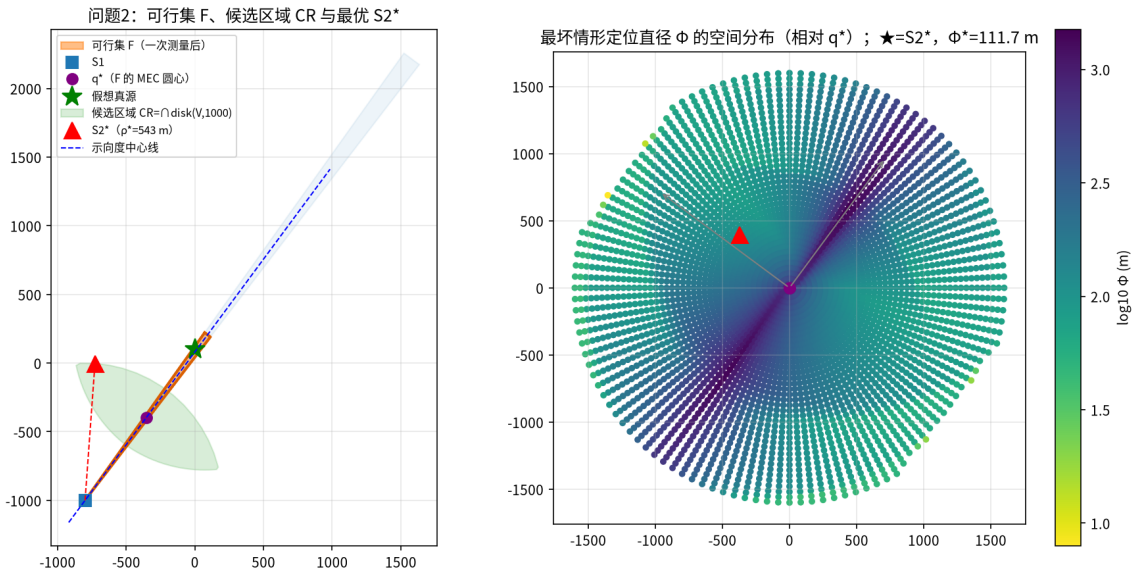


图 3: 问题二：可行集 F、保证接收交集 CR 与最优第二测点 S_2^*。

问题二结论：算例的最优第二测点为 $S_2^* = (-727.8, -2.9)$，对应剩余区域最坏直径 **111.7 m**（一次测量尺度的 **7.4%**）。该值是加密离散样本意义下的最坏值最优，其收敛性经边界与内部联合采样复核；垂直交会仅作为定性先验进入候选生成，不作为最优性依据。

5.3 问题三：全向源的搜索、定位与清除

5.3.1 覆盖层：九点布局与发现保证

全向源的可接收区域是圆盘，故测点集只需对目标圆域形成半径不超过 1000 m 的覆盖。采用中心加半径 1000 m 正八边形共 9 个测点：中心覆盖半径 1000 m 的内圆；圆域边界上最坏点位于相邻环点角平分方向，其到最近环点的距离

$$\sqrt{1800^2 + 1000^2 - 2 \cdot 1800 \cdot 1000 \cdot \cos 22.5^\circ} \;\approx\; 956 \text{ m} \;<\; 1000 \text{ m}, \tag{8}$$

故任意全向源（接收半径 ≥ 1000 m）至少被一个测点检出，发现保证成立（图 4）。需要说明的是，该保证只是单站的：边界上与环点同方位的源（如 $1800\mathbf{u}(0^\circ)$）距相邻环点约 1302 m，超出接收半径下界，此时仅单站检出，可行集为 2° 楔形，定位由补测管线继续收敛完成，不依赖双站交会。

式 (8) 的最坏点结论可以严格化而不仅是逐点验证。设边界点 $P(t) = 1800\mathbf{u}(t)$，$t \in [0^\circ, 45^\circ]$ 落在两相邻环点（方位 0° 与 45°，半径 1000 m）之间的扇区。$P(t)$ 到其中一个环点的距离

$$f(t) = \sqrt{1800^2 + 1000^2 - 2 \cdot 1800 \cdot 1000 \cos t}$$

的被开方数对 t 的导数为 $2 \cdot 1800 \cdot 1000 \sin t \geq 0$，故 f 在 $[0^\circ, 45^\circ]$ 上单调递增；点 $P(t)$ 到最近环点的距离为 $\min\{f(t), f(45^\circ - t)\}$，由对称性与单调性，其最大值在 $f(t) = f(45^\circ - t)$

即 $t = 22.5°$ 处取得，代入得 956 m。覆盖余量 $1000 - 956 = 43.6$ m，且该余量对扇区内所有边界点成立，使最坏点位于角平分方向这一结论从直觉升级为单调性论证。

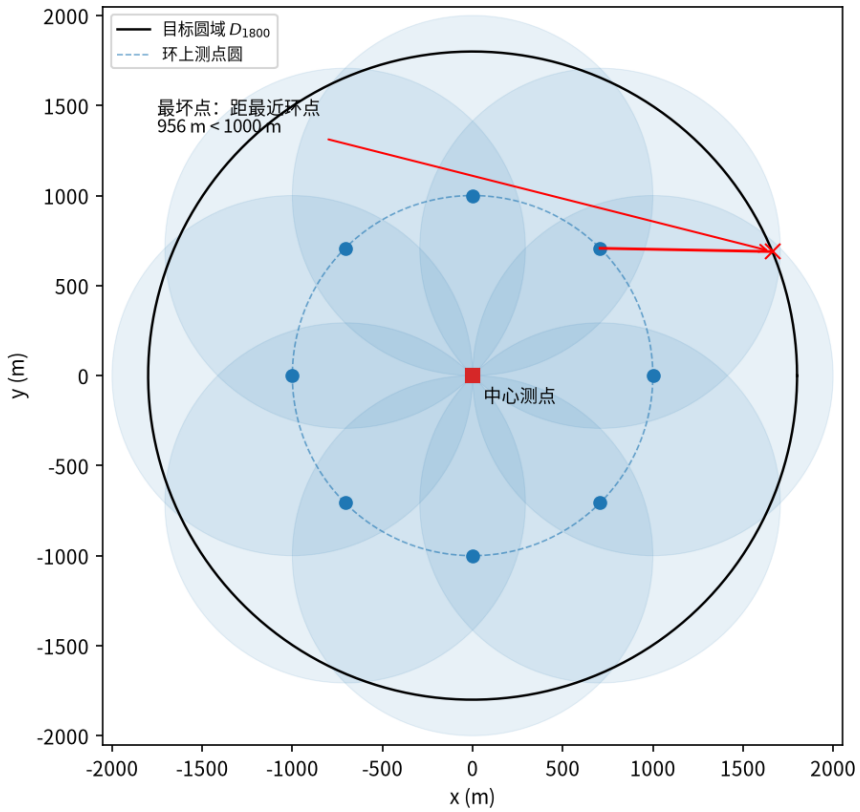


图 4: 问题三九点布局：8 个 1000 m 接收盘对目标圆域的覆盖与边界最坏点几何。

5.3.2 定位与清除层

每个测点按蛇形顺序扫描全部信道。对检出信号信道 c，用半平面交维护可行集并计算最小覆盖圆（MEC）。收敛闸门取

$$r_{\mathrm{MEC}} \;\le\; 19.75\ \mathrm{m} \;=\; \texttt{CLEAR_RADIUS} - 0.25\ \mathrm{m}\,: \tag{9}$$

20 m 为清除半径，0.25 m 裕度覆盖可行域多边形近似与浮点误差。$r_{\mathrm{MEC}} \le 19.75$ m 时圆心到真实源距离上界即 19.75 m，清除必然成功，这是清除环节唯一的确定性判据。

补测点取当前 MEC 圆心：清除点本就是收敛后的圆心，在圆心测向等于顺路，且一次方向读数给出半宽约 $r_{\mathrm{MEC}} \tan 1°$ 的窄带，收敛速度优于在 450 m 视线垂直点补测。该设计经配对消融实验验证（同批 200 局）：平均每源耗时 402.0 → 359.4 s，下降 10.6%；不宣称圆心补测理论最优。连续 `no_signal` 时启用折半逼近回退分支。收敛且机器狗距圆心不超过 400 m 时就地清除（收敛即清，闸值敏感性见 6.3 节）；全部收敛源按最近邻初解加 2-opt 局部改进排序清除（Croes, 1958）；2-opt 只保证局部最优，实测路径长度已非瓶颈。

5.3.3 调度层与异常降级

三项调度措施：**全局交错补测**，即补测阶段每步从所有未收敛信道选距当前位置最近的候选点，消除按信道序号折返；**跳测判定**，即已检出信道的可行集若完全在当前测点 1500 m 之外（接收半径上界），该次测量必然 `no_signal`，直接跳过；已清除信道亦跳测。该判定可证明安全（内点距离偏小只会少跳、不会误跳），配对 100 局平均每局省 176.5 s（-3.3%）。**异常降级**，即所有动作校验 `accepted` 标志；动作被拒或网络重试耗尽时按阶段降级：搜索中断保留已有示向度继续，单信道补测失败只放弃该信道，清除失败与时间不足的残留信道全部计入 `failed_channels` 上报，保证 $\text{discovered} = \text{cleared} \cup \text{failed}$ 的分类完备性，不弃局。故障注入回归 100 项（对 `measure`/`clear`/`exit` 注入拒绝与网络耗尽两类故障）全部通过。

5.3.4 结果

离线回归（md5 确定性可复现）：配对 200 局加全新 300 局共 500/500 全清，平均每源 359.4 s；相同随机案例下，采用 MEC 圆心补测与全局交错调度的策略较垂直侧移补测对照策略的 402.0 s/源降低 10.6%。官方模拟器真机演练 12 局 163/163 全清，其中采用本文最终策略的 6 局每源 $295.1 \sim 348.5$ s；正式测试 3 局 40/40 全清，平均 367.1 s/源（逐局数据见表 5）。虚拟时间结构经演练日志反推核实：每次测量恰 6 s、移动 5 m/s、清除约 6 s；六局正式测试的完整时间构成分解见图 10：移动占 $71\% \sim 81\%$、测量占 $17\% \sim 28\%$、清除不足 2%，与移动主导、布局优先的建模判断一致。图 5 还原了正式测试第三局（16 源）的完整动作轨迹：机器狗先沿九点布局完成扫描，随后按全局交错补测逐源收敛、清除，153 次测量与 16 次清除的位置序列与本节策略描述一一对应。

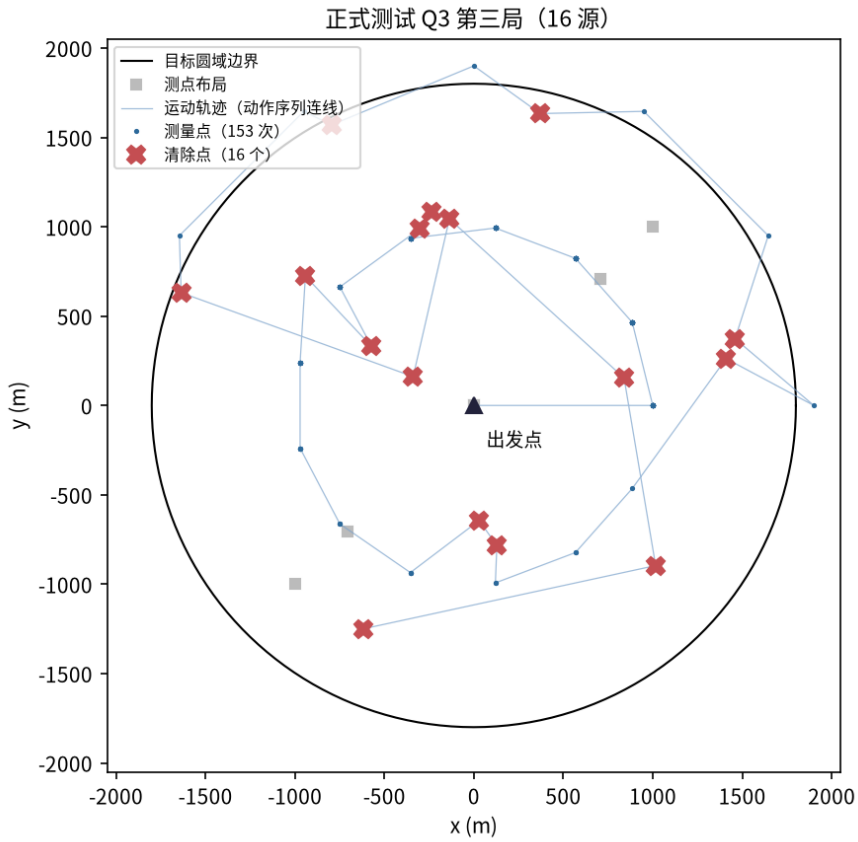


图 5: 正式测试 Q3 第三局（16 源）真实动作轨迹（由日志 169 个动作坐标还原）：测量点、清除点与运动路径。

5.4 问题四：混合源的搜索、定位与清除

5.4.1 保证层级结构

定向源的加入使发现成为非平凡问题，我们把问题四的保证组织为四层，并严格区分确定性保证与实验验证两类命题：

- **①发现层（确定性保证）**：26 点环形骨架对任意源位置、任意发射朝向保证检出；
- **②常规清除层（确定性判据）**：MEC 半径 $\leq 19.75\,\mathrm{m}$ 时出手必中；
- **③极端兜底层（几何确定性）**：$17.2\,\mathrm{m}$ 三角细网格，在补测用尽且 MEC 仍未收敛的病态分支下，候选网格中必存在可接收且可成功清除的测点；在线程序在剩余时间允许时执行该扫描；
- **④效率调度层（不承担保证责任）**：交错补测、顺路清除、跳测等效率措施。

发现后能全部定位并清除在 ①② 正常工作的前提下不属于定理范围，由大规模仿真与官方模拟器实测验证；论文与答辩表述为：确定性保证发现，病态情形有兜底保证，定位清除经仿真与实测验证，不使用确保清除所有源一类表述。四层在执行上的关系是：第 ③ 层只在补测用尽且 MEC 未收敛的分支上触发，正常情形零触发。

5.4.2 发现层：26 点环形骨架与弧判据

定向源可检出当且仅当存在测点 S 使 $|S-G| \le R$ 且 $(S-G)\cdot\mathbf{u}(\varphi) \ge 0$。对固定源位置 G，记其 1000 m 内可用测点的方位集合为 $\Theta(G)$，则对任意朝向 φ 均可检出的条件等价于

$$\max \mathrm{Gap}\big(\Theta(G)\big) \;\le\; 180^\circ, \tag{10}$$

其中等号对应测点恰落在半圆边界的情形；本文数值结果均有约 3.2° 的严格正裕度，故开、闭边界约定不改变布局结论。

式 (10) 的等价性证明：固定 G，记可用测点（$|S-G| \le R$）的方位集合为 $\Theta(G)$。题目的接收半圆包含边界，因此某一朝向不可检出，当且仅当全部可用测点严格落在其接收半平面之外；在圆周方位上，这等价于存在一段大于 180° 的空弧。取逆否并对朝向量化，任意朝向均可检出当且仅当最大方位空隙不超过 180°。本文扫描得到的最坏值为 176.21°，远离等号边界。

搜索骨架取**原点 + 13 点 @1000 m**（27.7° 间距）+ **12 点 @1900 m**（30° 间距）共 26 点：内环覆盖朝内与切向定向源，外环覆盖贴边朝外源，贴边最坏构型（源在 1800 m 边界、朝向径向向外、位于两外环点正中）的半平面裕度为 $1900\cos 15^\circ - 1800 = 35.2\,\mathrm{m} > 0$（图 6）。1900 m 环测点在目标圆域外，其合规依据是：题目正文以 1800 m 圆限制干扰源分布区域，附件 1 明确机器狗位置不构成约束；且数学上该外环不可省略：D_{1800} 边界点无法被任何完全场内点集的凸包包含。

无盲区验证（三重独立 + 分层闭合）：式 (10) 是含 max 的非凸判据，封闭形式的分情形证明繁琐且易漏，因此先用三种独立离散方式交叉复核，再用分层 Lipschitz 界处理格点之间与判据跳变边界：(i) 精确穷举（朝向解析精确，网格 0.5 m × 0.25°）：盲区 0 格，最小角裕度 3.198°，最坏最近距 585.2 m；(ii) 独立实现复核（直角网格 5 m，407 455 个源位置）：盲区 0，最坏空隙 $176.21^\circ < 180^\circ$；(iii) 边界薄缝检查：离散验证的共同理论缺口在判据的不连续点，即测点恰好出界的 $|S-G| = 1000\,\mathrm{m}$ 边界。对全部 26 个边界圆外侧 $0.02 \sim 2$ m、每圆 36 000 个方位点直查，最坏角裕度 3.218°。三套验证互证：薄缝不存在，零盲区结论在判据不连续点上同样成立（图 7）。三套验证的口径差异见表 4：采样方式与朝向处理互不相同，最坏值却落在同一量级，这正是独立验证的含义。

表 4: 发现层无盲区的三重独立验证口径对照（最坏结果两两独立取得）

验证通道	源位置采样 / 朝向处理	最坏结果
(i) 精确穷举	0.5 m 网格 × 0.25° 方位；朝向解析精确	盲区 0 格；最小角裕度 3.198°；最坏最近距 585.2 m
(ii) 独立实现复核	直角 5 m 网格（407 455 点）；弧判据（式 (10)）	最坏空隙 $176.21^\circ < 180^\circ$
(iii) 边界薄缝直查	26 个边界圆外侧 0.02 ∼ 2 m；每圆 36 000 方位点	最坏角裕度 3.218°

从离散扫描到连续域结论（分层 Lipschitz 闭合）：在可用测点集不变的开区域内，$\Theta(G)$ 中每个方位随 G 移动的漂移速率不超过 $1/d$（d 为该测站到 G 的距离），于是对采样格点 G_0 与半格距 h 邻域内的任意 G，

$$\left|\max \mathrm{Gap}(G) - \max \mathrm{Gap}(G_0)\right| \;\le\; \delta\Big(\frac{1}{d_i} + \frac{1}{d_j}\Big) \;\le\; \frac{2h}{d_{\mathrm{end}}(G_0)}, \tag{11}$$

其中 $d_{\mathrm{end}}(G_0)$ 为最大空隙两端测站到 G_0 距离的较小者。按可用集变化和端点退化把 D_{1800} 分四层：**(1) 主体区**用 0.5 m 网格检查 4027 万点，最小闭包量 $+3.32°$；**(2) 26 个检出圆的 ± 0.4 m 条带**用 0.02 m×0.02° 网格检查 1123 万点，跳变两侧分别闭合，最小闭包量 $+3.12°$；**(3) 14 个场内测站的 30 m 邻域**检查 6048 万点。近站端点由邻接弧解析界处理：外环可用站相对朝外空段中轴的偏离不超过约 19°，故相邻空隙不超过 120°、裕度至少 60°；其余端点距源不小于 450 m，采样最小裕度 51.4°，层内漂移不超过 0.012°；**(4) 全部 361 个圆–圆或圆–域界交点的 ± 0.65 m 邻域**用 0.005 m 网格检查 2289 万点，最小闭包量 $+3.19°$。四层覆盖目标圆域且闭包量均为正，故 $\max \mathrm{Gap}(G) < 180°$ 在连续域处处成立，保守最小角裕度下界约为 **3.1**°。全部点数、极值位置和闭包量由同一程序写入 `q4_closure_pointwise.json`，最终标志为 `all_closed=true`。

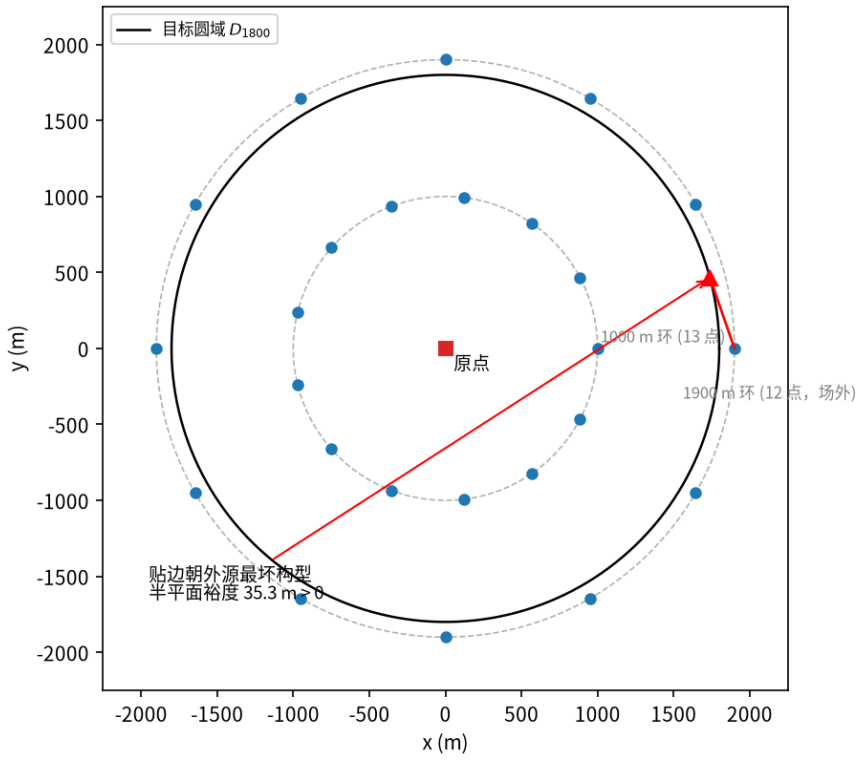


图 6: 问题四 26 点环形骨架与贴边朝外源最坏构型（半平面裕度 35.2 m）。

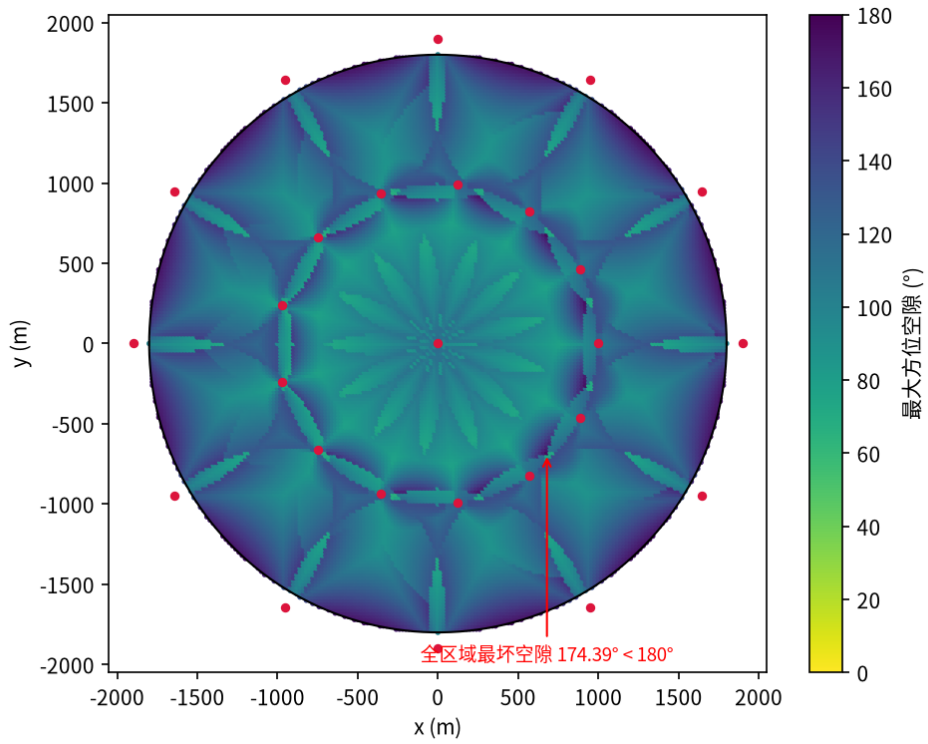


图 7: 弧判据多分辨率扫描：20 m 网格最坏值为 174.4°，5 m 网格为 176.2°，检出圆条带加密扫描为 176.8°；各离散验证均未发现盲区。

5.4.3 常规清除与效率调度

发现后的定位、清除与调度沿用问题三框架：楔形交可行集、MEC 收敛闸 19.75 m、MEC 圆心补测、深度阶梯与垂直交会候选、折半逼近回退、全局交错补测、收敛即清、顺路清除（绕路上限 400 m）与跳测判定。`no_signal` 的三义性（空信道、超半径、覆盖外）使其不能用于新增楔形，仅作不可用于定位的观测记录。

5.4.4 极端兜底层：17.2 m 三角细网格

某信道 16 次补测用尽且 $r_{\mathrm{MEC}} > 19.75\,\mathrm{m}$ 时，进入细网格兜底：对该信道正测向可行集外扩 20 m 的区域铺设间距 17.2 m 的三角格，按蛇形序逐点测量、遇信号就地清除。其保证由夹逼不等式给出：对任意源位置 G 与任意朝向 $\mathbf{u}(\varphi)$，理想点 $y = G + 10\mathbf{u}(\varphi)$ 距 G 恒为 10 m，三角格覆盖半径 $\rho = 17.2/\sqrt{3} \approx 9.93\,\mathrm{m}$，故存在格点 s 使 $|s - y| \le \rho$，从而

$$|s - G| \;\le\; 10 + 9.93 \;=\; 19.93\ \mathrm{m} \;<\; 20\ \mathrm{m} \quad \text{（清除必成功）}, \tag{12}$$

$$(s - G)\cdot\mathbf{u}(\varphi) \;\ge\; 10 - \rho \;=\; 0.07\ \mathrm{m} \;>\; 0 \quad \text{（}s\text{ 在前向半平面内，测量必有信号）}. \tag{13}$$

两个不等式对 G 与朝向均无假设，故兜底层提供的是**几何完备性**：候选网格中必存在一个既能收到信号、又处于清除半径内的测点；若该网格被完整扫描，则必能到达该点。程序执行层面，该层受时间保护（预留 180 s，不足则中止并在 summary 如实上报），即在线程序在剩余真实时间允许时执行该兜底策略；**正常情形零触发**：800 局离线回归与 10 局真机实验均未触发；强制触发单元测试（把补测上限压至 1 次）中触发即清除，验证了不

等式在本模拟器语义下的有效性：补测用尽的信道均能正确转入细网格兜底流程，并在检测到满足条件的网格点后完成清除，该分支责任闭环完整。

5.4.5 布局选型：搜索布局比较

为选择发现层布局，比较三种候选：36 点 6×6@720 m 方格、27 点等边三角格网与 26 点环形骨架。27 点三角格在混合 200 局中较 36 点方格慢 7.5%（855.0 对 795.1 s/源），测点数更少的三角格网反而更慢，说明发现层不能孤立地以测点数评价。在同一套二三四层、仅更换发现层的条件下，对 36 点方格与 26 点环形骨架各跑离线配对 400 局：26 点环形平均每源 595.4 s，较 36 点方格的 729.1 s 快 **18.3%**；全定向压力 100 局快 14.9%、全向压力 100 局快 24.3%。真机同源数配对方向一致：11 源局 841.0 → 644.1 s/源（−23.4%），13 源局 695.7 → 598.5 s/源（−14.0%），两种布局全清率无差异、兜底均零触发。

机制解释：稀疏骨架的价值取决于补测管线强度。26 点环形把负担从扫描期（巡视路程 25 709 m 降至 18 678 m、扫描测量 720 次降至 520 次，均为确定性节省）转移到补测期（每源示向度更少）；补测管线较弱时该代价无法吸收（27 点三角格情形），管线足够强（交错补测、收敛即清、跳测齐备）时红利即可兑现。36 点与 27 点三角格的对照混合了布局差异与管线差异，不能外推到 26 点环形。故问题四采用 26 点环形布局（图 8）。

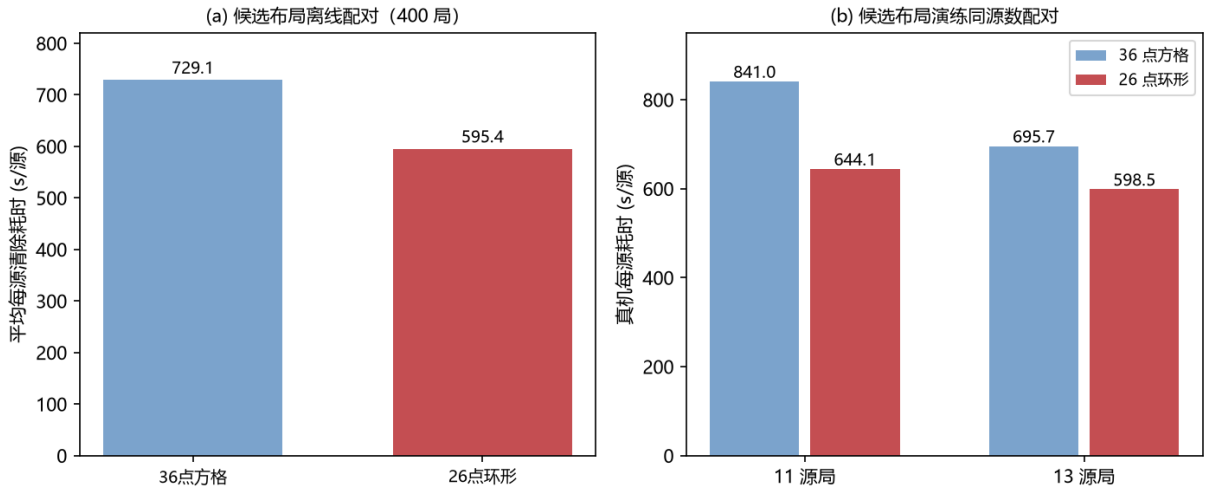


图 8: 36 点方格与 26 点环形两种候选布局的对照：(a) 400 局离线配对回归；(b) 模拟器演练中的同源数配对。

5.4.6 结果

离线回归：26 点环形布局混合 400 局 400/400 全清、平均 595.4 s/源；全定向 100 局 668.6 s/源、全向 100 局 513.5 s/源（均全清）。在相同定位、清除与调度机制下，26 点环形较 36 点方格的 729.1 s/源降低 18.3%，对照见图 8。模拟器演练 15 局 200/200 全清，其中采用 26 点环形布局的 3 局每源 527.1 ∼ 644.1 s；正式测试 3 局 37/37 全清，平均 550.2 s/源（逐局数据见表 5）。全部正式测试均正常退出，零拒绝动作、零失败信道，兜底层保持休眠。图 9 还原正式测试第二局（13 源）的动作轨迹：内环 1000 m 测点承担绝

大部分检出，外环 1900 m 测点仅用于覆盖贴边朝外源，291 次测量与 13 次清除的位置序列与本节策略描述一致。

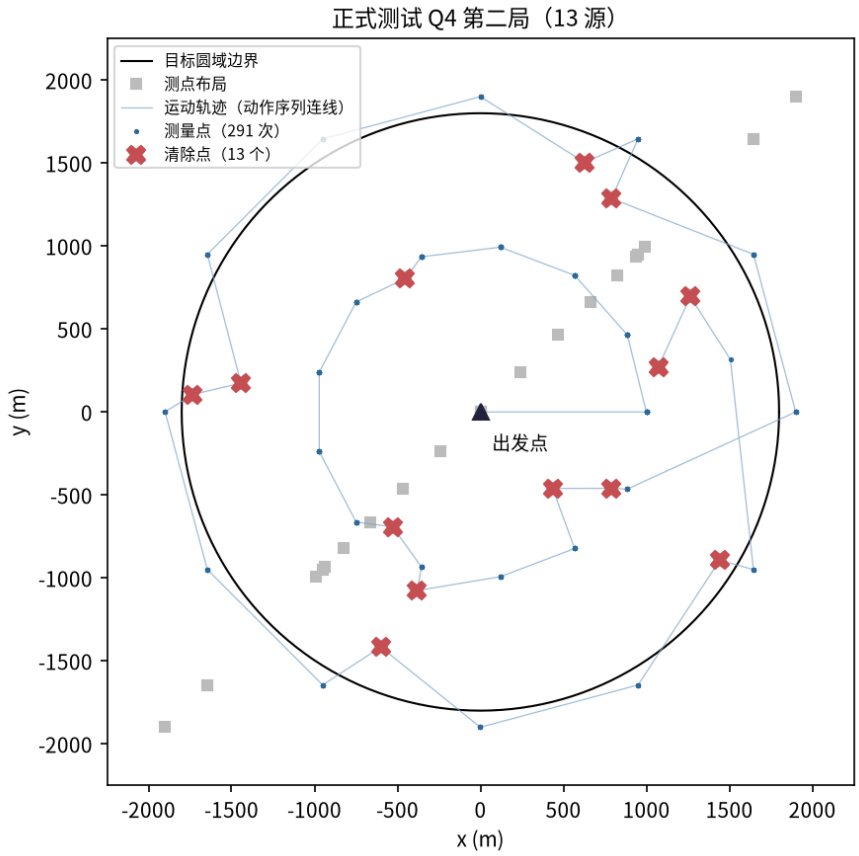


图 9: 正式测试 Q4 第二局（13 源）真实动作轨迹（由日志 304 个动作坐标还原）：灰色方块为含 1900 m 场外环的完整骨架。

六、模型的检验与敏感性分析

6.1 虚拟时间模型的核实

由演练日志反推的时间模型（测量 6 s、移动 5 m/s、清除约 6 s）在 33 局真机实验中保持一致：真机每源耗时与离线回归值吻合（问题三正式 367.1 对离线 359.4 s/源；问题四正式 550.2 对离线 595.4 s/源）。这说明离线模拟器复现了官方模拟器的主要运行特征（关键动作计时一致、每源耗时处于相近区间），因而离线调参结论可以迁移；本文不主张两个模拟器在所有输入与异常路径上语义等价。对六局正式测试的动作日志逐动作分解虚拟时间（图 10）：六局移动占比均不低于 71%（最高 81%），清除合计不足 2%，时间预算几乎全部消耗在巡视移动上，这是覆盖层测点少而精与调度层压缩路程两项设计的共同经验依据。

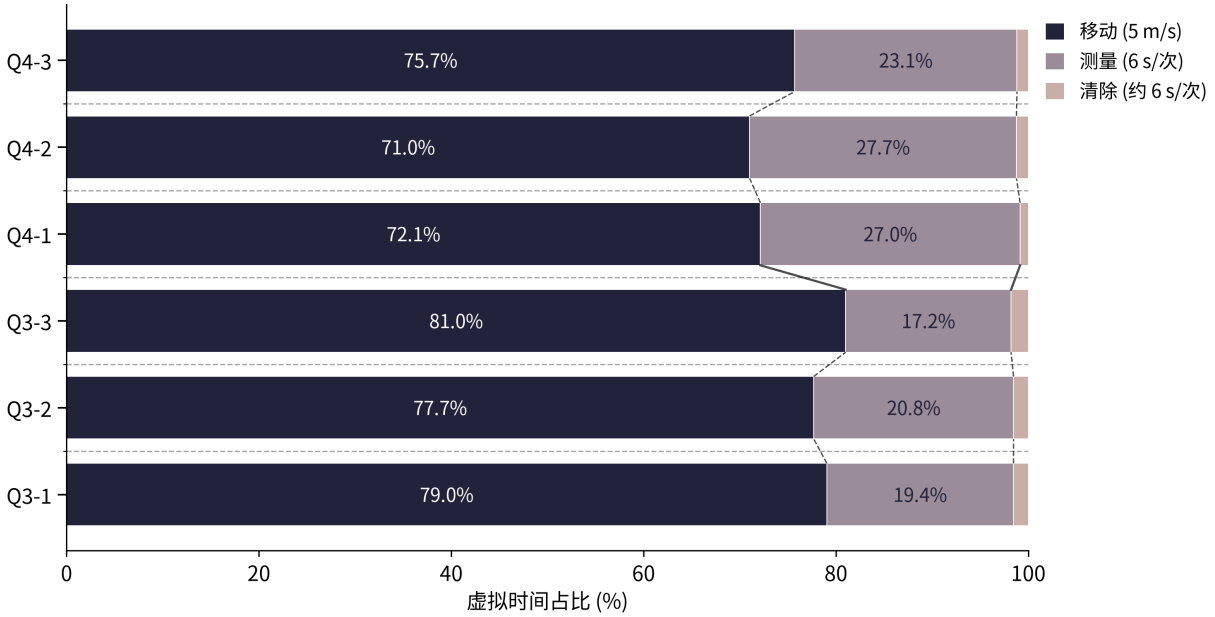


图 10: 六局正式测试的虚拟时间构成（由动作日志逐项分解）：移动占 71% ～ 81%，测量占 17% ～ 28%，清除不足 2%。

6.2 真机演练的逐局证据

除正式测试外，我们在官方模拟器上累计完成 27 局真机演练（问题三 12 局、问题四 15 局）。图 11 给出各局（本局源数，每源耗时）的散点，可读出两条与建模判断一致的经验规律：其一，**源数摊销**，即固定巡视成本被更多源分摊，每源耗时随本局源数增加整体下移（如问题三 12 源局 348.5 s 对 16 源局 304.8 s），这是移动主导的时间结构（图 10）的直接推论；其二，**波动尺度**，即相近源数的局间每源耗时波动约 ±10%，六局正式测试结果均落在相应波动带内，说明策略表现稳定（26 点环形布局对 36 点方格的真机同源数配对见 5.4.5 节）。

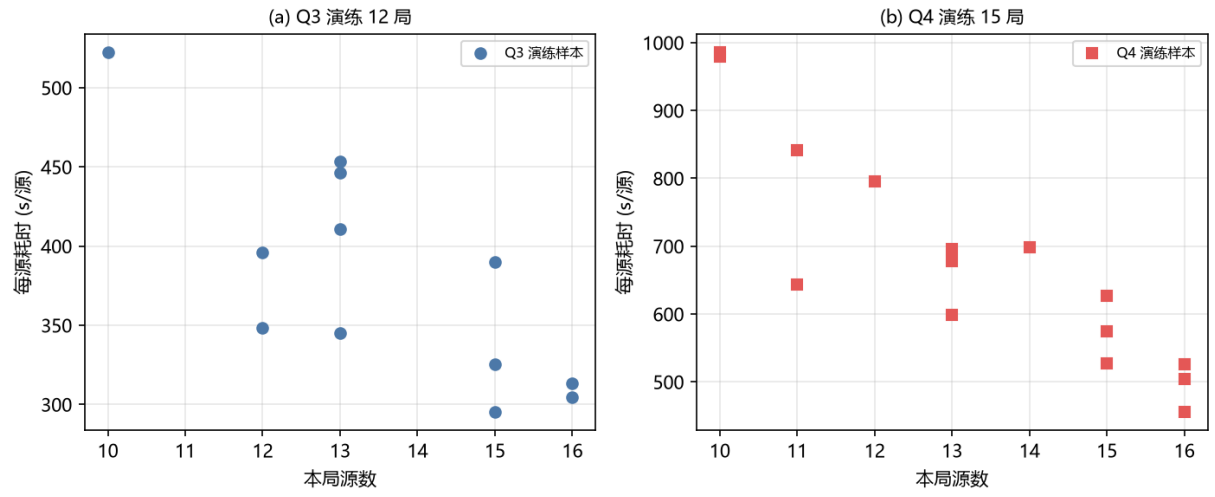


图 11: 27 局真机演练的逐局散点（数据自演练登记表）：每源耗时随本局源数增加被摊销下移，相近源数的局间波动约 ±10%。

6.3 阈值敏感性（配对种子、单变量）

所有敏感性实验均为配对种子离线实验：同一批随机种子下只改动单变量，差值可归因于该变量本身（图 12）。

- **Q3 收敛即清距离间**：闸值取 $0, 150, 400, 600, 800$ m 时每源耗时依次为 398.4、360.8、359.4、359.8、363.9 s，其中 $150 \sim 600$ m 为平台区，取值 400 m 距最优 0.4 s/源；禁用（0）代价 +10.9%。
- **Q4 顺路清除绕路上限**：上限取 $200, 300, 400, 600, 800$ m 时依次为 738.7、735.7、734.4、737.6、745.4 s，曲线呈光滑 U 形，谷底恰在取值 400 m 处。
- **Q4 补测几何参数**：深度、横移距离、侧序等 8 组变体的扰动影响在 $\pm 0.1\%$ 量级，补测几何远离敏感区。
- **Q4 布局**：见 5.4.5 节布局对照（-18.3%，真机 $-14\% \sim -23\%$）。

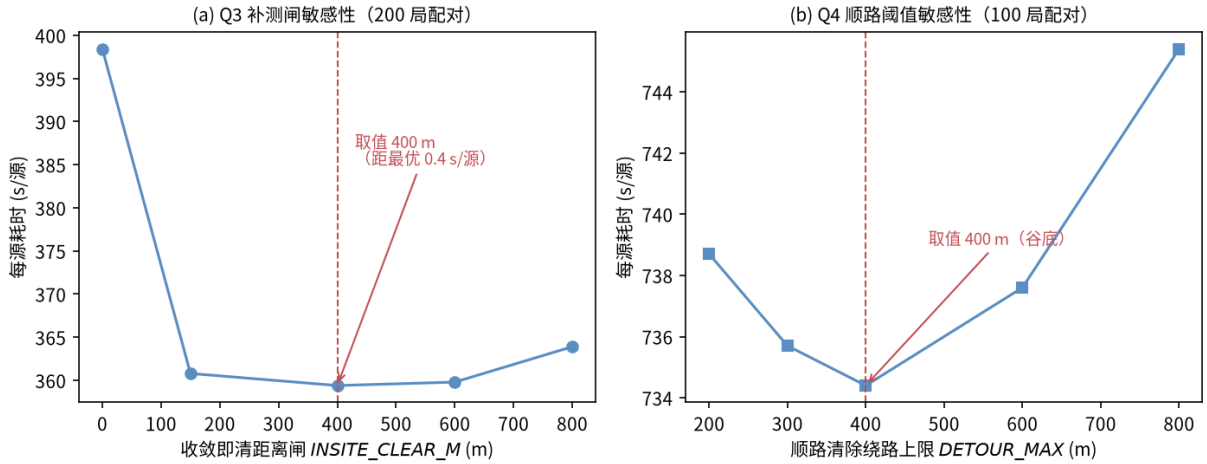


图 12: 两个关键阈值的配对种子敏感性扫描：均处于平台区/谷底，参数选择对扰动不敏感。

6.4 异常与故障回归

故障注入回归对 `measure/clear/exit` 三类接口注入动作被拒与网络重试耗尽两类故障（覆盖搜索、补测、兜底各阶段，共 100 项），断言分类完备性等不变式，全部通过；时间不足（`remaining = 1 s`）注入下程序正常降级退出、summary 完整。需要说明的验证边界：离线回归只累计虚拟时间，时间保护分支仅验证了正确进入，未在真实网络竞争下验证保护效果；单请求最坏真实耗时约 25 s（8 s 超时 $\times$ 3 次重试加退避），各阶段 $150 \sim 300$ s 的时间预留可容忍连续多次最坏请求。

6.5 正式测试结果

表 5: 正式测试逐局结果（2026-09-12；脚本 md5 与提交版本一致，脱敏逐局日志见支撑材料）

局	源数	清除	虚拟时间	每源耗时	测量/清除次数	真实耗时
Q3-1	12	12/12	4667.5 s	389.0 s/源	151/12	1.2 s
Q3-2	12	12/12	4679.2 s	389.9 s/源	162/12	1.2 s
Q3-3	16	16/16	5339.1 s	333.7 s/源	153/16	1.2 s
Q3 小计	**40**	**40/40**	—	**367.1 s/源**	—	—
Q4-1	10	10/10	6978.0 s	697.8 s/源	314/10	2.2 s
Q4-2	13	13/13	6294.7 s	484.2 s/源	291/13	1.8 s
Q4-3	14	14/14	7085.3 s	506.1 s/源	273/14	1.6 s
Q4 小计	**37**	**37/37**	—	**550.2 s/源**	—	—

六局全部被模拟器接受退出（exit accepted）、**零拒绝动作**、**零失败信道**；兜底层全程零触发。正式成绩与离线回归、演练水平的吻合度见表内数字与 6.1 节。

七、模型的评价与推广

7.1 主要优点

1. **保证与验证分层**：哪些结论是确定性命题（Thales 判据、弧判据、19.75 m 闸、细网格夹逼不等式）、哪些由实验验证（定位清除成功率），全文口径统一，不混用。
2. **参数选择全部有实验依据**：MEC 圆心补测（−10.6% 消融）、收敛即清闸（平台区）、顺路上限（U 形谷底）、布局（400 局配对 + 真机同源数配对）均为配对实验：实验中保持随机案例与其他模型组件一致，仅改动单一变量，减少场景差异对比较结果的干扰。
3. **工程鲁棒性**：异常降级语义使任何单点故障不弃局，故障注入 100 项验证；真机 33 局 440/440 的实测记录支撑结论。
4. **连续域证据链完整**：弧判据先经三条独立数值通道交叉复核，再按主体区、检出圆条带、测站邻域和边界交点邻域分层闭合；程序输出记录每层点数、最小闭包量和最终 `all_closed=true`。

7.2 主要缺点

1. **问题三无清除兜底层**：单信道 8 次补测用尽即计入 `failed_channels`，理论上存在近平行薄楔病态构型使定位收敛失败且无对应对物；800 局离线与真机均未触发，风险极低但机制上未封堵（问题四已用细网格封堵同类缺口）。

2. **问题四证明采用分层数值认证**：连续域结论依赖分层网格、Lipschitz 界与近站邻接弧解析界，而非一个简短的全解析闭式证明；复现成本较高，必须随附逐层日志和机器可读结果。

3. **问题二目标为样本意义**：Φ 的凸性未证明，最坏值靠边界与内部联合采样估计，报告的是加密离散样本下的最优，非连续域严格最优；严格化需区间分支定界。

4. **时间保护未在真实竞争下检验**：真实耗时保护分支未经历过网络持续劣化的实测，仅验证了逻辑正确与预算裕度。

7.3 改进方向

针对缺点 1，可把问题四的细网格兜底按信道可行域裁剪后移植到问题三（问题三全向源无需前向半平面约束，网格可更稀）；针对缺点 2，弧判据可对 26 点骨架的对称类做完整分情形解析证明，或采用经区间算术认证的分支定界；针对缺点 3，用区间分析对 Φ 做分支定界可得连续域严格上界。

AI 工具使用说明

本队按照《全国大学生数学建模竞赛人工智能工具使用规定（2026 年试行）》声明：竞赛期间，我们使用生成式人工智能工具辅助完成了以下工作：(1) 离线模拟器与验证脚本（回归测试、故障注入、布局覆盖检查）的代码编写与调试；(2) 数值实验的组织与结果整理；(3) 论文初稿的文字整理与排版。全部模型的建模思路、口径选择、判据推导、参数取舍与结论检验由参赛队完成并逐项复核；所有数值结果均可由支撑材料中的源程序与日志复现，AI 未生成任何未经我们验证的数据或结论。

八、参考文献

[1] BISHOP A N, FIDAN B, ANDERSON B D O, et al. Optimality analysis of sensor-target localization geometries[J]. Automatica, 2010, 46(3): 479-492.

[2] NARDONE S C, LINDGREN A G, GONG K F. Optimal observer motion for localization with bearing measurements[J]. Computers & Mathematics with Applications, 1989, 18(1-3): 171-180.

[3] ZHAO S, CHEN B M, LEE T H. Optimal placement of bearing-only sensors for target localization[C]//Proceedings of the American Control Conference. IEEE, 2012: 27-32.

[4] JAULIN L. Set-membership localization with probabilistic errors[J]. Robotics and Autonomous Systems, 2011, 59(6): 489-495.

[5] LIU Y S, ZHAO Y, WU F L, et al. Bearing-only target tracking based on ellipsoidal outer-bounding set-membership estimation[J]. Journal of Beijing University of Aeronautics and Astronautics, 2017, 43(3): 497-505.

[6] CHOSET H. Coverage for robotics—a survey of recent results[J]. Annals of Mathematics and Artificial Intelligence, 2001, 31(1-4): 113-126.

[7] GALCERAN E, CARRERAS M. A survey on coverage path planning for robotics[J]. Robotics and Autonomous Systems, 2013, 61(12): 1258-1276.

[8] GABRIELY Y, RIMON E. Competitive on-line coverage of grid environments by a mobile robot[J]. Computational Geometry, 2003, 24(3): 197-224.

[9] CROES G A. A method for solving traveling-salesman problems[J]. Operations Research, 1958, 6(6): 791-812.

[10] POPOVIĆ M, OTT J, RÜCKIN J, et al. Learning-based methods for adaptive informative path planning[J]. Robotics and Autonomous Systems, 2024, 174: 104627.

[11] LEE T K, BAEK S H, CHOI Y H, et al. Smooth coverage path planning and control of mobile robots based on high-resolution grid map representation[J]. Robotics and Autonomous Systems, 2011, 59(11): 801-812.

附录 A 支撑材料文件清单

支撑材料压缩包包含以下文件（与论文内容一致）：

- `robot_q3_v2.py`：问题三在线程序（正式测试版本，md5 b6631804）；
- `robot_q4.py`：问题四在线程序（正式测试版本，md5 c1c77732）；
- `q1_core.py`、`q1_run.py`：问题一判据与数值实验；
- `q2_model.py`：问题二鲁棒选点模型；
- `offline_test_v2.py`、`q4_offline.py`：问题三/四离线回归；
- `fault_injection_test.py`：故障注入回归（100 项）；
- `q4_layout_26_check.py`、`q4_boundary_sliver_check.py`、`q4_layout_tri_check.py`：布局验证与边界薄缝检查；
- `q4_strip_check.py`、`q4_closure_pointwise.py`：无盲区结论的四层采样与 Lipschitz 闭合检查（结果文件 `all_closed=true`）；
- `q4_fallback_trigger_test.py`：兜底强制触发单测；

- 逐局日志 `logs-anon/`（`robot-q3-formal-*-anon.jsonl`、`robot-q4-formal-*-anon.jsonl`，已去除 `robot_id` 字段以满足匿名评审要求）。

附录 B　主要源程序

以下为两问正式测试版本的完整源代码，与提交的支撑材料一致；按竞赛要求，附录中仅保留代码本身，所有注释与文档字符串已去除（支撑材料中的原件含完整注释，可对照阅读）。

B.1　问题三程序 `robot_q3_v2.py`

```python
from __future__ import annotations
import argparse
import json
import math
import time
import uuid
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterable, Protocol
from urllib.error import HTTPError, URLError
from urllib.request import Request, urlopen
ARENA_RADIUS = 1800.0
BEARING_ERROR_DEG = 1.0
CLEAR_RADIUS = 20.0
CHANNELS = tuple(range(1, 21))

@dataclass(frozen=True)
class Point:
    x: float
    y: float

    def distance(self, other: 'Point') -> float:
        return math.hypot(self.x - other.x, self.y - other.y)

@dataclass(frozen=True)
class Bearing:
    station: Point
    angle_deg: float

@dataclass
class ChannelState:
    bearings: list[Bearing] = field(default_factory=list)
    cleared: bool = False
    polygon: list[Point] = field(default_factory=list)

class RobotAPI(Protocol):

    def enter(self) -> dict:
        ...

    def measure(self, point: Point, channel: int) -> dict:
        ...
```

```python

    def clear(self, point: Point, channel: int) -> dict:
        ...

    def exit(self) -> dict:
        ...

class ActionRejected(RuntimeError):
    pass

def require_accepted(response: dict, action: str) -> None:
    if response.get('accepted') is not True:
        raise ActionRejected(f'{action} 未被接受：{response}')

class HttpRobotAPI:

    def __init__(self, base_url: str, robot_id: str, log_path: Path):
        self.base_url = base_url.rstrip('/')
        self.robot_id = robot_id
        self.log_path = log_path
        self.counter = 0

    def _id(self, prefix: str) -> str:
        self.counter += 1
        return f'{prefix}-{self.counter}-{uuid.uuid4().hex[:8]}'

    def _post(self, path: str, payload: dict) -> dict:
        body = json.dumps(payload, ensure_ascii=False).encode('utf-8')
        req = Request(self.base_url + path, data=body, headers={'Content-Type': 'application/json'},
↪ method='POST')
        last_error: Exception | None = None
        for attempt in range(3):
            try:
                with urlopen(req, timeout=8) as response:
                    result = json.loads(response.read().decode('utf-8'))
                self._log(path, payload, result)
                return result
            except (URLError, TimeoutError, HTTPError) as exc:
                last_error = exc
                if attempt < 2:
                    time.sleep(0.25 * 2 ** attempt)
        raise RuntimeError(f'请求 {path} 失败：{last_error}')

    def _base(self, request_id: str) -> dict:
        return {'arena_id': 'default', 'robot_id': self.robot_id, 'request_id': request_id}

    def _log(self, path: str, request: dict, response: dict) -> None:
        self.log_path.parent.mkdir(parents=True, exist_ok=True)
        row = {'path': path, 'request': request, 'response': response}
        with self.log_path.open('a', encoding='utf-8') as f:
            f.write(json.dumps(row, ensure_ascii=False) + '\n')

    def enter(self) -> dict:
        return self._post('/enter', self._base(self._id('enter')))

    def measure(self, point: Point, channel: int) -> dict:
```

```
        p = self._base(self._id('measure'))
        p.update({'position': {'x': point.x, 'y': point.y}, 'channel': channel})
        return self._post('/measure', p)

    def clear(self, point: Point, channel: int) -> dict:
        p = self._base(self._id('clear'))
        p.update({'position': {'x': point.x, 'y': point.y}, 'channel': channel})
        return self._post('/clear', p)

    def exit(self) -> dict:
        return self._post('/exit', self._base(self._id('exit')))

def regular_polygon(radius: float, sides: int=720) -> list[Point]:
    return [Point(radius * math.cos(2 * math.pi * i / sides), radius * math.sin(2 * math.pi * i / sides))
    ↪  for i in range(sides)]

def cross(ax: float, ay: float, bx: float, by: float) -> float:
    return ax * by - ay * bx

def clip_halfplane(poly: list[Point], station: Point, angle_deg: float, keep_left: bool) -> list[Point]:
    if not poly:
        return []
    a = math.radians(angle_deg)
    dx, dy = (math.cos(a), math.sin(a))

    def signed(p: Point) -> float:
        return cross(dx, dy, p.x - station.x, p.y - station.y)

    def inside(p: Point) -> bool:
        value = signed(p)
        return value >= -1e-08 if keep_left else value <= 1e-08
    result: list[Point] = []
    previous = poly[-1]
    previous_in = inside(previous)
    for current in poly:
        current_in = inside(current)
        if current_in != previous_in:
            sx, sy = (current.x - previous.x, current.y - previous.y)
            denominator = cross(dx, dy, sx, sy)
            if abs(denominator) > 1e-12:
                t = -signed(previous) / denominator
                result.append(Point(previous.x + t * sx, previous.y + t * sy))
        if current_in:
            result.append(current)
        previous, previous_in = (current, current_in)
    return result

def polygon_from_bearings(bearings: Iterable[Bearing]) -> list[Point]:
    poly = regular_polygon(ARENA_RADIUS)
    for bearing in bearings:
        poly = clip_halfplane(poly, bearing.station, bearing.angle_deg - BEARING_ERROR_DEG, keep_left=
    ↪ True)
        poly = clip_halfplane(poly, bearing.station, bearing.angle_deg + BEARING_ERROR_DEG, keep_left=
    ↪ False)
    return poly

```

```python
def min_dist_to_polygon(p: Point, poly: list[Point]) -> float:
    best = float('inf')
    n = len(poly)
    for i in range(n):
        a, b = (poly[i], poly[(i + 1) % n])
        abx, aby = (b.x - a.x, b.y - a.y)
        apx, apy = (p.x - a.x, p.y - a.y)
        denom = abx * abx + aby * aby
        t = 0.0 if denom <= 1e-12 else max(0.0, min(1.0, (apx * abx + apy * aby) / denom))
        d = math.hypot(p.x - (a.x + t * abx), p.y - (a.y + t * aby))
        if d < best:
            best = d
    return best

def circle_from_two(a: Point, b: Point) -> tuple[Point, float]:
    center = Point((a.x + b.x) / 2, (a.y + b.y) / 2)
    return (center, center.distance(a))

def circle_from_three(a: Point, b: Point, c: Point) -> tuple[Point, float] | None:
    d = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y))
    if abs(d) < 1e-09:
        return None
    aa, bb, cc = (a.x * a.x + a.y * a.y, b.x * b.x + b.y * b.y, c.x * c.x + c.y * c.y)
    ux = (aa * (b.y - c.y) + bb * (c.y - a.y) + cc * (a.y - b.y)) / d
    uy = (aa * (c.x - b.x) + bb * (a.x - c.x) + cc * (b.x - a.x)) / d
    center = Point(ux, uy)
    return (center, center.distance(a))

def covers(center: Point, radius: float, points: Iterable[Point]) -> bool:
    return all((center.distance(p) <= radius + 1e-06 for p in points))

def minimum_enclosing_circle(points: list[Point]) -> tuple[Point, float]:
    if not points:
        raise ValueError(' 空定位区域')
    if len(points) == 1:
        return (points[0], 0.0)
    best: tuple[Point, float] | None = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            candidate = circle_from_two(points[i], points[j])
            if covers(*candidate, points) and (best is None or candidate[1] < best[1]):
                best = candidate
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            for k in range(j + 1, len(points)):
                candidate = circle_from_three(points[i], points[j], points[k])
                if candidate and covers(*candidate, points) and (best is None or candidate[1] < best[1]):
                    best = candidate
    if best is None:
        raise RuntimeError(' 未能构造最小覆盖圆')
    return best

def discovery_points() -> list[Point]:
    points = [Point(0.0, 0.0)]
    for i in range(8):
        angle = 2 * math.pi * i / 8
```

```
        points.append(Point(1000 * math.cos(angle), 1000 * math.sin(angle)))
    return points

def serpentine_channels(index: int) -> tuple[int, ...]:
    return CHANNELS if index % 2 == 0 else tuple(reversed(CHANNELS))

def choose_refinement_point(center: Point, state: ChannelState, fallback_center: Point | None=None,
↪ az_offset_deg: float=0.0) -> Point:
    if fallback_center is not None:
        last = state.bearings[-1]
        return Point((fallback_center.x + center.x) / 2, (fallback_center.y + center.y) / 2)
    last = state.bearings[-1]
    angle = math.radians(last.angle_deg + 90.0 + az_offset_deg)
    step = 450.0
    candidates = [Point(center.x + step * math.cos(angle), center.y + step * math.sin(angle)), Point(
↪ center.x - step * math.cos(angle), center.y - step * math.sin(angle))]
    valid = [p for p in candidates if math.hypot(p.x, p.y) <= ARENA_RADIUS]
    if not valid:
        return center
    return min(valid, key=lambda p: p.distance(last.station))

def optimized_clear_order(start: Point, targets: dict[int, Point]) -> list[int]:
    remaining = set(targets)
    route: list[int] = []
    current = start
    while remaining:
        channel = min(remaining, key=lambda c: current.distance(targets[c]))
        route.append(channel)
        remaining.remove(channel)
        current = targets[channel]

    def length(order: list[int]) -> float:
        if not order:
            return 0.0
        total = start.distance(targets[order[0]])
        total += sum((targets[a].distance(targets[b]) for a, b in zip(order, order[1:])))
        return total
    improved = True
    best = length(route)
    while improved:
        improved = False
        for i in range(len(route) - 1):
            for j in range(i + 1, len(route)):
                candidate = route[:i] + list(reversed(route[i:j + 1])) + route[j + 1:]
                value = length(candidate)
                if value + 1e-06 < best:
                    route, best, improved = (candidate, value, True)
    return route

class Q3Strategy:
    CLEAR_RESERVE_S = 150.0
    REFINE_RESERVE_S = 300.0
    INSITE_CLEAR_M = 400.0

    def __init__(self, api: RobotAPI):
        self.api = api
```

```
        self.states = {channel: ChannelState() for channel in CHANNELS}
        self.deadline: float | None = None
        self.t0: float = 0.0

    def time_left(self) -> float:
        if self.deadline is None:
            return float('inf')
        return self.deadline - time.monotonic()

    def run(self) -> dict:
        self.t0 = time.monotonic()
        entered = self.api.enter()
        if entered.get('accepted') is not True:
            raise RuntimeError(f'/enter 未被接受：{entered}')
        remaining = entered.get('remaining_real_duration_s')
        if isinstance(remaining, (int, float)) and remaining > 0:
            self.deadline = self.t0 + float(remaining)
        errors: list[str] = []
        search_points = discovery_points()
        current_point = Point(0.0, 0.0)
        try:
            for point_index, point in enumerate(search_points):
                for channel in serpentine_channels(point_index):
                    state = self.states[channel]
                    if state.cleared:
                        continue
                    if state.bearings:
                        state.polygon = polygon_from_bearings(state.bearings)
                        if state.polygon and min_dist_to_polygon(point, state.polygon) > 1500.0:
                            continue
                    response = self.api.measure(point, channel)
                    current_point = point
                    require_accepted(response, f' 测量（点 {point_index}，频道 {channel}）')
                    result = response.get('measure_result')
                    if result == 'direction':
                        self.states[channel].bearings.append(Bearing(point, float(response['svd_deg'])))
                    elif result == 'near':
                        clear_response = self.api.clear(point, channel)
                        require_accepted(clear_response, f' 清除（点 {point_index}，频道 {channel}）')
                        self.states[channel].cleared = clear_response.get('clear_result') == 'success'
                if self.time_left() < self.REFINE_RESERVE_S + self.CLEAR_RESERVE_S:
                    break
        except (ActionRejected, RuntimeError) as exc:
            errors.append(f' 覆盖搜索中断：{exc}')
        ready: dict[int, Point] = {}
        failed: list[int] = []
        pending = {c for c, s in self.states.items() if s.bearings and (not s.cleared)}
        attempts = {c: 0 for c in pending}
        fallback_from: dict[int, Point] = {}
        try:
            while pending and self.time_left() >= self.REFINE_RESERVE_S:
                candidates: dict[int, tuple[Point, Point, float]] = {}
                finished: list[int] = []
                for channel in pending:
                    state = self.states[channel]
                    state.polygon = polygon_from_bearings(state.bearings)
```

```
                    if not state.polygon:
                        failed.append(channel)
                        finished.append(channel)
                        continue
                    center, radius = minimum_enclosing_circle(state.polygon)
                    if radius <= CLEAR_RADIUS - 0.25:
                        if current_point.distance(center) <= self.INSITE_CLEAR_M:
                            try:
                                clr = self.api.clear(center, channel)
                                require_accepted(clr, f' 就地清除 (频道 {channel})')
                                if clr.get('clear_result') == 'success':
                                    state.cleared = True
                                else:
                                    ready[channel] = center
                            except (ActionRejected, RuntimeError) as exc:
                                errors.append(f' 频道 {channel} 就地清除失败: {exc}')
                                ready[channel] = center
                        else:
                            ready[channel] = center
                        finished.append(channel)
                        continue
                    if attempts[channel] >= 8:
                        failed.append(channel)
                        finished.append(channel)
                        continue
                    if channel in fallback_from:
                        point = choose_refinement_point(center, state, fallback_center=fallback_from[
    ↪ channel])
                    else:
                        point = Point(center.x, center.y)
                    candidates[channel] = (point, center, radius)
                for channel in finished:
                    pending.discard(channel)
                if not candidates:
                    break
                channel = min(candidates, key=lambda c: current_point.distance(candidates[c][0]))
                next_point = candidates[channel][0]
                state = self.states[channel]
                try:
                    response = self.api.measure(next_point, channel)
                    require_accepted(response, f' 补测 (频道 {channel})')
                except (ActionRejected, RuntimeError) as exc:
                    errors.append(f' 频道 {channel} 补测失败: {exc}')
                    failed.append(channel)
                    pending.discard(channel)
                    continue
                current_point = next_point
                attempts[channel] += 1
                result = response.get('measure_result')
                if result == 'direction':
                    state.bearings.append(Bearing(next_point, float(response['svd_deg'])))
                    fallback_from.pop(channel, None)
                elif result == 'near':
                    try:
                        clear_response = self.api.clear(next_point, channel)
                        require_accepted(clear_response, f' 清除 (频道 {channel})')
```

```
                    except (ActionRejected, RuntimeError) as exc:
                        errors.append(f' 频道 {channel} 清除失败: {exc}')
                        failed.append(channel)
                        pending.discard(channel)
                        continue
                    state.cleared = clear_response.get('clear_result') == 'success'
                    if not state.cleared:
                        failed.append(channel)
                    pending.discard(channel)
                elif result == 'no_signal':
                    fallback_from[channel] = next_point
                else:
                    errors.append(f' 频道 {channel} 未知测量结果: {response}')
                    failed.append(channel)
                    pending.discard(channel)
        except (ActionRejected, RuntimeError) as exc:
            errors.append(f' 定位阶段中断: {exc}')
        for channel in pending:
            if not self.states[channel].cleared and channel not in ready:
                failed.append(channel)
        clear_failed: list[int] = []
        clear_order = optimized_clear_order(current_point, ready)
        for channel in clear_order:
            if self.time_left() <= self.CLEAR_RESERVE_S * 0.3:
                break
            clear_point = ready.pop(channel)
            try:
                response = self.api.clear(clear_point, channel)
                require_accepted(response, f' 清除 (频道 {channel})')
            except (ActionRejected, RuntimeError) as exc:
                errors.append(f' 频道 {channel} 清除失败: {exc}')
                clear_failed.append(channel)
                continue
            current_point = clear_point
            self.states[channel].cleared = response.get('clear_result') == 'success'
            if not self.states[channel].cleared:
                clear_failed.append(channel)
        clear_failed.extend(ready.keys())
        summary = {'discovered_channels': [c for c, s in self.states.items() if s.bearings or s.cleared],
↪  'cleared_channels': [c for c, s in self.states.items() if s.cleared], 'failed_channels': sorted((c
↪ for c in set(failed + clear_failed) if not self.states[c].cleared)), 'unfound_channels': sorted((c for
↪  c, s in self.states.items() if not s.bearings and (not s.cleared))), 'real_time_used_s': round(time.
↪ monotonic() - self.t0, 1), 'time_left_s': round(max(self.time_left(), 0.0), 1)}
        if errors:
            summary['action_errors'] = errors
        try:
            exit_response = self.api.exit()
            require_accepted(exit_response, '/exit')
            summary['exit'] = exit_response
            summary['exit_accepted'] = True
        except (ActionRejected, RuntimeError) as exc:
            summary['exit'] = {'error': str(exc)}
            summary['exit_accepted'] = False
        return summary

def main() -> None:
```

```
    parser = argparse.ArgumentParser()
    parser.add_argument('--live', action='store_true', help=' 连接官方模拟器；默认不连接')
    parser.add_argument('--robot-id', default='', help=' 当前登录的参赛队号')
    parser.add_argument('--base-url', default='http://127.0.0.1:2026')
    parser.add_argument('--log', default='robot-actions.jsonl')
    args = parser.parse_args()
    if not args.live:
        print('dry-run：未连接模拟器。使用 --live --robot-id < 参赛队号> 才会执行测试。')
        return
    if not args.robot_id:
        raise SystemExit('live 模式必须提供 --robot-id')
    api = HttpRobotAPI(args.base_url, args.robot_id, Path(args.log))
    print(json.dumps(Q3Strategy(api).run(), ensure_ascii=False, indent=2))
if __name__ == '__main__':
    main()
```

B.2 问题四程序 robot_q4.py

```
from __future__ import annotations
import argparse
import json
import math
import time
from pathlib import Path
from robot_q3_v2 import ARENA_RADIUS, BEARING_ERROR_DEG, CHANNELS, CLEAR_RADIUS, ActionRejected, Point, Bearing, ChannelState, HttpRobotAPI, clip_halfplane, polygon_from_bearings, min_dist_to_polygon, minimum_enclosing_circle, regular_polygon, require_accepted, optimized_clear_order

def ring_points(n: int, r: float, phase_deg: float=0.0) -> list[Point]:
    return [Point(r * math.cos(2 * math.pi * i / n + math.radians(phase_deg)), r * math.sin(2 * math.pi * i / n + math.radians(phase_deg))) for i in range(n)]

def search_points() -> list[Point]:
    pts = [Point(0.0, 0.0)]
    for k in range(13):
        a = 2.0 * math.pi * k / 13.0
        pts.append(Point(1000.0 * math.cos(a), 1000.0 * math.sin(a)))
    for k in range(12):
        a = 2.0 * math.pi * k / 12.0
        pts.append(Point(1900.0 * math.cos(a), 1900.0 * math.sin(a)))
    return pts

def search_points_tri27() -> list[Point]:
    pts = [(-1500.0, -1800.0), (-500.0, -1800.0), (500.0, -1800.0), (1500.0, -1800.0), (-2000.0, -933.9746), (-1000.0, -933.9746), (0.0, -933.9746), (1000.0, -933.9746), (2000.0, -933.9746), (-2500.0, -67.9492), (-1500.0, -67.9492), (-500.0, -67.9492), (500.0, -67.9492), (1500.0, -67.9492), (2500.0, -67.9492), (-2000.0, 798.0762), (-1000.0, 798.0762), (0.0, 798.0762), (1000.0, 798.0762), (2000.0, 798.0762), (-1500.0, 1664.1016), (-500.0, 1664.1016), (500.0, 1664.1016), (1500.0, 1664.1016), (-1000.0, 2530.127), (0.0, 2530.127), (1000.0, 2530.127)]
    order = [(-500.0, -67.9492), (-1500.0, -67.9492), (-2500.0, -67.9492), (-2000.0, 798.0762), (-1000.0, 798.0762), (-1500.0, 1664.1016), (-1000.0, 2530.127), (0.0, 2530.127), (1000.0, 2530.127), (1500.0, 1664.1016), (500.0, 1664.1016), (-500.0, 1664.1016), (0.0, 798.0762), (500.0, -67.9492), (1000.0, 798.0762), (1500.0, -67.9492), (2000.0, 798.0762), (2500.0, -67.9492), (2000.0, -933.9746), (1000.0, -933.9746), (0.0, -933.9746), (-1000.0, -933.9746), (-2000.0, -933.9746), (-1500.0, -1800.0), (-500.0, -1800.0), (500.0, -1800.0), (1500.0, -1800.0)]
    pts_set = {tuple(p) for p in pts}
```

```
    order_set = {tuple(p) for p in order}
    assert pts_set == order_set and len(order) == len(pts), ' 巡视路径与测点集不一致'
    return [Point(x, y) for x, y in order]

def serpentine_channels(index: int) -> tuple[int, ...]:
    return CHANNELS if index % 2 == 0 else tuple(reversed(CHANNELS))
DETOUR_MAX = 400.0
FINE_GRID_SPACING = 17.2
FINE_GRID_PAD = 20.0
FINE_GRID_RADIUS = 1820.0
FINE_GRID_RANGE_CLIP = 1520.0
FINE_GRID_RESERVE_S = 180.0

def _point_in_convex_poly(p: Point, poly: list[Point]) -> bool:
    pos = neg = False
    for i in range(len(poly)):
        a = poly[i]
        b = poly[(i + 1) % len(poly)]
        cross = (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x)
        if cross > 1e-09:
            pos = True
        elif cross < -1e-09:
            neg = True
    return not (pos and neg)

def fine_grid_points(bearings: list[Bearing]) -> list[Point]:
    if not bearings:
        return []
    station = bearings[0].station
    poly = polygon_from_bearings(bearings)
    if not poly or len(poly) < 3:
        return []
    angle = math.radians(bearings[0].angle_deg)
    ex = (math.cos(angle), math.sin(angle))
    ey = (-math.sin(angle), math.cos(angle))
    sx, sy = (station.x, station.y)
    local = [((p.x - sx) * ex[0] + (p.y - sy) * ex[1], (p.x - sx) * ey[0] + (p.y - sy) * ey[1]) for p in
↪ poly]
    pad = FINE_GRID_PAD
    xmin = min((p[0] for p in local)) - pad
    xmax = max((p[0] for p in local)) + pad
    ymin = min((p[1] for p in local)) - pad
    ymax = max((p[1] for p in local)) + pad
    spacing = FINE_GRID_SPACING
    gap = spacing * math.sqrt(3.0) / 2.0
    raw: list[tuple[int, float, Point]] = []
    for row in range(math.floor(ymin / gap) - 1, math.ceil(ymax / gap) + 2):
        ly = row * gap
        for col in range(math.floor(xmin / spacing - 0.5 * row) - 1, math.ceil(xmax / spacing - 0.5 * row
↪ ) + 2):
            lx = spacing * (col + 0.5 * row)
            px = sx + lx * ex[0] + ly * ey[0]
            py = sy + lx * ex[1] + ly * ey[1]
            if px * px + py * py > FINE_GRID_RADIUS * FINE_GRID_RADIUS + 1e-09:
                continue
            if math.hypot(px - sx, py - sy) > FINE_GRID_RANGE_CLIP:
```

```
                continue
            p = Point(px, py)
            if _point_in_convex_poly(p, poly) or min_dist_to_polygon(p, poly) <= pad + 1e-09:
                raw.append((row, lx, p))
    ordered: list[Point] = []
    for index, row in enumerate(sorted({item[0] for item in raw})):
        row_points = sorted((item for item in raw if item[0] == row), key=lambda item: item[1], reverse=
↪ bool(index % 2))
        ordered.extend((item[2] for item in row_points))
    return ordered

class Q4Strategy:
    CLEAR_RESERVE_S = 150.0
    REFINE_RESERVE_S = 300.0
    MAX_REFINE_ITERS = 16

    def __init__(self, api):
        self.api = api
        self.states = {channel: ChannelState() for channel in CHANNELS}
        self.search_ready: dict[int, Point] = {}
        self.deadline: float | None = None
        self.t0: float = 0.0
        self.deferred_errors: list[str] = []
        self.fine_grid_channels: list[int] = []

    def time_left(self) -> float:
        if self.deadline is None:
            return float('inf')
        return self.deadline - time.monotonic()

    def coverage_search(self) -> Point:
        current = Point(0.0, 0.0)
        self.current_pos = current
        for point_index, point in enumerate(search_points()):
            for channel in serpentine_channels(point_index):
                state = self.states[channel]
                if state.cleared or channel in self.search_ready:
                    continue
                discovered_count = sum((bool(s.bearings) or s.cleared for s in self.states.values()))
                if discovered_count >= 16 and (not state.bearings):
                    continue
                if state.bearings:
                    state.polygon = polygon_from_bearings(state.bearings)
                    if state.polygon and min_dist_to_polygon(point, state.polygon) > 1500.0:
                        continue
                response = self.api.measure(point, channel)
                current = point
                self.current_pos = current
                require_accepted(response, f' 测量 (点 {point_index}, 频道 {channel})')
                result = response.get('measure_result')
                if result == 'direction':
                    state.bearings.append(Bearing(point, float(response['svd_deg'])))
                    state.polygon = polygon_from_bearings(state.bearings)
                    if state.polygon:
                        center, radius = minimum_enclosing_circle(state.polygon)
                        if radius <= CLEAR_RADIUS - 0.25:
```

```
                            self.search_ready[channel] = center
                elif result == 'near':
                    clear_response = self.api.clear(point, channel)
                    require_accepted(clear_response, f' 清除 (点 {point_index}, 频道 {channel})')
                    state.cleared = clear_response.get('clear_result') == 'success'
            discovered_count = sum((bool(s.bearings) or s.cleared for s in self.states.values()))
            if discovered_count >= 16 and all((s.cleared or c in self.search_ready for c, s in self.
    ↪ states.items() if s.bearings)):
                break
            sp = search_points()
            if point_index + 1 < len(sp):
                nxt = sp[point_index + 1]
                for channel, center in list(self.search_ready.items()):
                    if self.states[channel].cleared:
                        continue
                    detour = point.distance(center) + center.distance(nxt) - point.distance(nxt)
                    if detour <= DETOUR_MAX:
                        try:
                            clr = self.api.clear(center, channel)
                            require_accepted(clr, f' 顺路清除 (频道 {channel})')
                            if clr.get('clear_result') == 'success':
                                self.states[channel].cleared = True
                                self.search_ready.pop(channel, None)
                        except (ActionRejected, RuntimeError) as exc:
                            self.deferred_errors.append(f' 顺路清除 (频道 {channel}) 失败: {exc}')
                            break
            if self.time_left() < self.REFINE_RESERVE_S + self.CLEAR_RESERVE_S:
                break
        return current

    def _ladder_candidates(self, state: ChannelState) -> list[Point]:
        b = state.bearings[0]
        a = math.radians(b.angle_deg)
        ux, uy = (math.cos(a), math.sin(a))
        nx, ny = (-uy, ux)
        pts = []
        for t in (0.0, 350.0, 700.0, 1050.0):
            for s in (450.0, -450.0):
                pts.append(Point(b.station.x + t * ux + s * nx, b.station.y + t * uy + s * ny))
        return pts

    def refine(self, channel: int) -> tuple[bool, Point | None]:
        state = self.states[channel]
        if not state.bearings:
            return (False, None)
        state.polygon = polygon_from_bearings(state.bearings)
        if not state.polygon:
            return (False, None)
        center, radius = minimum_enclosing_circle(state.polygon)
        if radius <= CLEAR_RADIUS - 0.25:
            return (True, center)
        no_signal_streak = 0
        az_offset = 0.0
        candidate: Point | None = None
        last_ok: Point | None = state.bearings[-1].station
        visited: list[tuple[float, float]] = []
```

```
        ladder = self._ladder_candidates(state)
        li = 0
        walk_k = 0
        for _ in range(self.MAX_REFINE_ITERS):
            state.polygon = polygon_from_bearings(state.bearings)
            if not state.polygon:
                return (False, None)
            center, radius = minimum_enclosing_circle(state.polygon)
            if radius <= CLEAR_RADIUS - 0.25:
                return (True, center)
            if self.time_left() < self.REFINE_RESERVE_S:
                break

            def fresh(p: Point) -> Point:
                for extra in range(0, 360, 30):
                    q = p if extra == 0 else self._perp_candidate(center, state, az_offset + extra)
                    if (q.x, q.y) not in visited:
                        return q
                return p
            if li < len(ladder) and radius > 100.0:
                candidate = fresh(ladder[li])
                li += 1
            elif no_signal_streak == 0:
                candidate = fresh(self._perp_candidate(center, state, az_offset))
            elif radius <= 150.0 and last_ok is not None:
                walk_k += 1
                dx, dy = (center.x - last_ok.x, center.y - last_ok.y)
                seg = math.hypot(dx, dy)
                if seg < 1e-06:
                    candidate = fresh(center)
                else:
                    nx, ny = (-dy / seg, dx / seg)
                    delta = max(30.0, min(120.0, 2.0 * radius))
                    side = (1.0, -1.0, 0.0)[(walk_k - 1) % 3]
                    base = Point(last_ok.x + 0.5 * dx, last_ok.y + 0.5 * dy)
                    candidate = fresh(Point(base.x + side * delta * nx, base.y + side * delta * ny))
            elif no_signal_streak % 2 == 1:
                candidate = fresh(Point((candidate.x + center.x) / 2, (candidate.y + center.y) / 2))
            else:
                az_offset += 60.0
                candidate = fresh(self._perp_candidate(center, state, az_offset))
            visited.append((candidate.x, candidate.y))
            self.current_pos = candidate
            response = self.api.measure(candidate, channel)
            require_accepted(response, f' 补测 (频道 {channel})')
            result = response.get('measure_result')
            if result == 'direction':
                state.bearings.append(Bearing(candidate, float(response['svd_deg'])))
                no_signal_streak = 0
                last_ok = candidate
            elif result == 'near':
                clear_response = self.api.clear(candidate, channel)
                require_accepted(clear_response, f' 清除 (频道 {channel})')
                state.cleared = clear_response.get('clear_result') == 'success'
                return (state.cleared, candidate if not state.cleared else None)
            else:
```

```
                no_signal_streak += 1
        state.polygon = polygon_from_bearings(state.bearings)
        if state.polygon:
            center, radius = minimum_enclosing_circle(state.polygon)
            if radius <= CLEAR_RADIUS - 0.25:
                response = self.api.clear(center, channel)
                require_accepted(response, f' 兜底清除 (频道 {channel})')
                if response.get('clear_result') == 'success':
                    state.cleared = True
                    return (True, None)
            elif self.time_left() >= FINE_GRID_RESERVE_S:
                if self.fine_grid_clear(channel):
                    return (True, None)
        return (False, None)

    def fine_grid_clear(self, channel: int) -> bool:
        state = self.states[channel]
        if not state.bearings or state.cleared:
            return False
        points = fine_grid_points(state.bearings)
        self.fine_grid_channels.append(channel)
        if not points:
            self.deferred_errors.append(f' 频道 {channel} 细网格兜底无候选点')
            return False
        for index, p in enumerate(points):
            if self.time_left() < FINE_GRID_RESERVE_S * 0.5:
                self.deferred_errors.append(f' 频道 {channel} 细网格兜底因时间保护中止 ({index}/{len(
↪ points)})')
                return False
            response = self.api.measure(p, channel)
            require_accepted(response, f' 细网格兜底 (频道 {channel})')
            result = response.get('measure_result')
            if result in ('direction', 'near'):
                clear_response = self.api.clear(p, channel)
                require_accepted(clear_response, f' 细网格兜底清除 (频道 {channel})')
                if clear_response.get('clear_result') == 'success':
                    state.cleared = True
                    return True
                self.deferred_errors.append(f' 频道 {channel} 细网格清除失败 (信号点)')
        return False

    @staticmethod
    def _perp_candidate(center: Point, state: ChannelState, az_offset: float) -> Point:
        last = state.bearings[-1]
        angle = math.radians(last.angle_deg + 90.0 + az_offset)
        step = 450.0
        candidates = [Point(center.x + step * math.cos(angle), center.y + step * math.sin(angle)), Point(
↪ center.x - step * math.cos(angle), center.y - step * math.sin(angle))]
        valid = [p for p in candidates if math.hypot(p.x, p.y) <= ARENA_RADIUS]
        if not valid:
            return center
        return min(valid, key=lambda p: p.distance(last.station))

    def run(self) -> dict:
        self.t0 = time.monotonic()
        entered = self.api.enter()
```

```
        if entered.get('accepted') is not True:
            raise RuntimeError(f'/enter 未被接受: {entered}')
        remaining = entered.get('remaining_real_duration_s')
        if isinstance(remaining, (int, float)) and remaining > 0:
            self.deadline = self.t0 + float(remaining)
        errors: list[str] = []
        self.current_pos = Point(0.0, 0.0)
        try:
            self.coverage_search()
        except (ActionRejected, RuntimeError) as exc:
            errors.append(f' 覆盖搜索中断: {exc}')
        current = self.current_pos
        ready: dict[int, Point] = dict(self.search_ready)
        failed: list[int] = []
        pending: dict[int, dict] = {}
        for channel, state in self.states.items():
            if state.cleared or not state.bearings or channel in ready:
                continue
            state.polygon = polygon_from_bearings(state.bearings)
            if not state.polygon:
                failed.append(channel)
                continue
            center, radius = minimum_enclosing_circle(state.polygon)
            if radius <= CLEAR_RADIUS - 0.25:
                ready[channel] = center
                continue
            pending[channel] = dict(li=0, attempts=0, no_signal_streak=0, az_offset=0.0, walk_k=0, kind='
    ↪ none', az_dedup=0.0, last_ok=state.bearings[-1].station, last_cand=None, visited=[], ladder=self.
    ↪ _ladder_candidates(state))
        try:
            exhausted: list[int] = []
            while pending and self.time_left() >= self.REFINE_RESERVE_S:
                candidates: dict[int, Point] = {}
                finished: list[int] = []
                for channel, st in pending.items():
                    state = self.states[channel]
                    state.polygon = polygon_from_bearings(state.bearings)
                    if not state.polygon:
                        failed.append(channel)
                        finished.append(channel)
                        continue
                    center, radius = minimum_enclosing_circle(state.polygon)
                    if radius <= CLEAR_RADIUS - 0.25:
                        try:
                            clr = self.api.clear(center, channel)
                            require_accepted(clr, f' 就地清除 (频道 {channel})')
                            if clr.get('clear_result') == 'success':
                                state.cleared = True
                                self.search_ready.pop(channel, None)
                            else:
                                ready[channel] = center
                        except (ActionRejected, RuntimeError) as exc:
                            errors.append(f' 频道 {channel} 就地清除失败: {exc}')
                            ready[channel] = center
                        finished.append(channel)
                        continue
```

```
                    if st['attempts'] >= self.MAX_REFINE_ITERS:
                        finished.append(channel)
                        exhausted.append(channel)
                        continue
                    if st['li'] < len(st['ladder']) and radius > 100.0:
                        cand, kind = (st['ladder'][st['li']], 'ladder')
                    elif st['no_signal_streak'] == 0:
                        cand = self._perp_candidate(center, state, st['az_offset'])
                        kind = 'perp'
                    elif radius <= 150.0 and st['last_ok'] is not None:
                        dx, dy = (center.x - st['last_ok'].x, center.y - st['last_ok'].y)
                        seg = math.hypot(dx, dy)
                        if seg < 1e-06:
                            cand, kind = (center, 'none')
                        else:
                            nx, ny = (-dy / seg, dx / seg)
                            delta = max(30.0, min(120.0, 2.0 * radius))
                            side = (1.0, -1.0, 0.0)[st['walk_k'] % 3]
                            base = Point(st['last_ok'].x + 0.5 * dx, st['last_ok'].y + 0.5 * dy)
                            cand = Point(base.x + side * delta * nx, base.y + side * delta * ny)
                            kind = 'walk'
                    elif st['no_signal_streak'] % 2 == 1 and st['last_cand']:
                        cand = Point((st['last_cand'].x + center.x) / 2, (st['last_cand'].y + center.y) /
    ↪ 2)
                        kind = 'bisect'
                    else:
                        cand = self._perp_candidate(center, state, st['az_offset'] + 60.0)
                        kind = 'az'
                    st['kind'] = kind
                    st['az_dedup'] = st['az_offset'] + (60.0 if kind == 'az' else 0.0)
                    for extra in range(0, 360, 30):
                        q = cand if extra == 0 else self._perp_candidate(center, state, st['az_dedup'] +
    ↪ extra)
                        if (q.x, q.y) not in st['visited']:
                            cand = q
                            break
                    candidates[channel] = cand
                for channel in finished:
                    pending.pop(channel)
                if not candidates:
                    break
                channel = min(candidates, key=lambda c: current.distance(candidates[c]))
                cand = candidates[channel]
                st = pending[channel]
                if st['kind'] == 'ladder':
                    st['li'] += 1
                elif st['kind'] == 'walk':
                    st['walk_k'] += 1
                elif st['kind'] == 'az':
                    st['az_offset'] += 60.0
                st['visited'].append((cand.x, cand.y))
                st['last_cand'] = cand
                st['attempts'] += 1
                current = cand
                self.current_pos = cand
                response = self.api.measure(cand, channel)
```

```
                require_accepted(response, f' 补测 (频道 {channel})')
                result = response.get('measure_result')
                if result == 'direction':
                    st['last_ok'] = cand
                    self.states[channel].bearings.append(Bearing(cand, float(response['svd_deg'])))
                    st['no_signal_streak'] = 0
                elif result == 'near':
                    clear_response = self.api.clear(cand, channel)
                    require_accepted(clear_response, f' 清除 (频道 {channel})')
                    self.states[channel].cleared = clear_response.get('clear_result') == 'success'
                    if not self.states[channel].cleared:
                        failed.append(channel)
                    pending.pop(channel)
                else:
                    st['no_signal_streak'] += 1
        except (ActionRejected, RuntimeError) as exc:
            errors.append(f' 补测阶段中断: {exc}')
        for channel in list(dict.fromkeys(list(pending) + exhausted)):
            state = self.states[channel]
            state.polygon = polygon_from_bearings(state.bearings)
            if state.polygon:
                center, radius = minimum_enclosing_circle(state.polygon)
                if radius <= CLEAR_RADIUS - 0.25:
                    try:
                        response = self.api.clear(center, channel)
                        require_accepted(response, f' 兜底清除 (频道 {channel})')
                        if response.get('clear_result') == 'success':
                            state.cleared = True
                    except (ActionRejected, RuntimeError) as exc:
                        errors.append(f' 频道 {channel} 兜底失败: {exc}')
                elif self.time_left() >= FINE_GRID_RESERVE_S:
                    try:
                        self.fine_grid_clear(channel)
                    except (ActionRejected, RuntimeError) as exc:
                        errors.append(f' 频道 {channel} 细网格兜底失败: {exc}')
            if not state.cleared:
                failed.append(channel)
            pending.pop(channel, None)
        current = self.current_pos
        retry: list[int] = []
        clear_order = optimized_clear_order(current, ready)
        while clear_order and self.time_left() > self.CLEAR_RESERVE_S * 0.3:
            channel = clear_order.pop(0)
            clear_point = ready.pop(channel)
            try:
                response = self.api.clear(clear_point, channel)
                require_accepted(response, f' 清除 (频道 {channel})')
            except (ActionRejected, RuntimeError) as exc:
                errors.append(f' 频道 {channel} 清除失败: {exc}')
                failed.append(channel)
                continue
            current = clear_point
            if response.get('clear_result') == 'success':
                self.states[channel].cleared = True
            else:
                retry.append(channel)
```

```
        for channel in retry:
            if self.time_left() < self.CLEAR_RESERVE_S * 0.3:
                break
            try:
                ok, clear_point = self.refine(channel)
            except (ActionRejected, RuntimeError) as exc:
                errors.append(f' 频道 {channel} 回炉失败: {exc}')
                failed.append(channel)
                continue
            if ok and clear_point is not None:
                try:
                    response = self.api.clear(clear_point, channel)
                    require_accepted(response, f' 清除 (频道 {channel})')
                except (ActionRejected, RuntimeError) as exc:
                    errors.append(f' 频道 {channel} 清除失败: {exc}')
                    failed.append(channel)
                    continue
                current = clear_point
                if response.get('clear_result') == 'success':
                    self.states[channel].cleared = True
                else:
                    failed.append(channel)
            else:
                failed.append(channel)
        failed.extend(ready.keys())
        failed.extend(retry)
        errors.extend(self.deferred_errors)
        summary = {'discovered_channels': [c for c, s in self.states.items() if s.bearings or s.cleared],
 'cleared_channels': [c for c, s in self.states.items() if s.cleared], 'failed_channels': sorted((c
for c in set(failed) if not self.states[c].cleared)), 'unfound_channels': sorted((c for c, s in self.
states.items() if not s.bearings and (not s.cleared))), 'fine_grid_fallback': sorted(self.
fine_grid_channels), 'real_time_used_s': round(time.monotonic() - self.t0, 1), 'time_left_s': round(
max(self.time_left(), 0.0), 1)}
        if errors:
            summary['action_errors'] = errors
        try:
            exit_response = self.api.exit()
            require_accepted(exit_response, '/exit')
            summary['exit'] = exit_response
            summary['exit_accepted'] = True
        except (ActionRejected, RuntimeError) as exc:
            summary['exit'] = {'error': str(exc)}
            summary['exit_accepted'] = False
        return summary

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument('--live', action='store_true', help=' 连接官方模拟器; 默认不连接')
    parser.add_argument('--robot-id', default='', help=' 当前登录的参赛队号')
    parser.add_argument('--base-url', default='http://127.0.0.1:2026')
    parser.add_argument('--log', default='robot-q4-actions.jsonl')
    args = parser.parse_args()
    if not args.live:
        print('dry-run: 未连接模拟器。使用 --live --robot-id < 参赛队号> 才会执行测试。')
        return
    if not args.robot_id:
```

```
515         raise SystemExit('live 模式必须提供 --robot-id')
516     api = HttpRobotAPI(args.base_url, args.robot_id, Path(args.log))
517     print(json.dumps(Q4Strategy(api).run(), ensure_ascii=False, indent=2))
518 if __name__ == '__main__':
519     main()
```